
Panel de estadísticas y logros para el juez
'¡Acepta el reto!'
Statistics and achievements panel for the online
judge '¡Acepta el reto!'



Trabajo de Fin de Grado
Curso 2025–2026

Autor

Néstor García Mayor
María Pajares Vázquez
Inés Triviño Rello

Directores

Pedro Pablo Gómez Martín
Marco Antonio Gómez Martín

Grado en Ingeniería de Software
Facultad de Informática
Universidad Complutense de Madrid



Panel de estadísticas y logros para el juez
'¡Acepta el reto!'
Statistics and achievements panel for the
online judge '¡Acepta el reto!'

Trabajo de Fin de Grado en Ingeniería de Software

Autor

Néstor García Mayor
María Pajares Vázquez
Inés Triviño Rello

Directores

Pedro Pablo Gómez Martín
Marco Antonio Gómez Martín

Convocatoria: *Junio 2026*

Grado en Ingeniería de Software
Facultad de Informática
Universidad Complutense de Madrid

DIA de MES de AÑO

Dedicatoria

Texto

Agradecimientos

Texto

Resumen

Panel de estadísticas y logros para el juez '¡Acepta el reto!'

Un resumen en castellano de media página, incluyendo el título en castellano. A continuación, se escribirá una lista de no más de 10 palabras clave.

Palabras clave

Máximo 10 palabras clave separadas por comas

Abstract

Statistics and achievements panel for the online judge '¡Acepta el reto!'

An abstract in English, half a page long, including the title in English. Below, a list with no more than 10 keywords.

Keywords

10 keywords max., separated by commas.

Contents

1	Introducción	1
1.1	Motivación y antecedentes	1
1.2	Objetivos	2
1.3	Plan de trabajo	3
2	Estado del Arte de Jueces en Línea	5
2.1	Introducción	5
2.2	Categorización de jueces de programación	6
2.3	Gamificación	7
2.4	Diseño y arquitectura	10
2.5	Conclusiones	11
3	Estado del Arte de Tecnologías para Desarrollo Web	13
3.1	Introducción	13
3.2	Frontend	14
3.3	Backend	15
3.4	Conclusión	17
4	Diseño de la Aplicación	19
4.1	Introducción	19
4.2	Análisis de requisitos	20
4.2.1	Estadísticas de usuario	20
4.2.2	Estadísticas de ejercicios	21
4.2.3	Sistema de logros	21
4.2.4	Tabla de clasificación	21
4.2.5	Compartición de estadísticas y logros	22
4.3	Diseño del sistema de gamificación	22
4.3.1	Introducción a la gamificación	22
4.3.2	Análisis de jugadores y mecánicas	23
4.3.3	Propuesta de diseño	25
4.4	Diseño de la interfaz	31
4.5	Arquitectura del sistema	32

5	Implementación	35
6	Evaluación	37
7	Conclusiones y Trabajo Futuro	39
	Contribuciones Personales	41
	Bibliography	43
A	Especificación de Requisitos Software	45
A.1	Introducción	45
A.1.1	Propósito del documento	45
A.1.2	Ámbito del producto	45
A.1.3	Definiciones, acrónimos y abreviaciones	46
A.1.4	Referencias	46
A.2	Descripción general del producto	46
A.2.1	Perspectiva del producto	46
A.2.2	Funciones del producto	46
A.2.3	Limitaciones del producto	47
A.2.4	Características de los usuarios	47
A.2.5	Supuestos y dependencias	47
A.3	Requisitos	47
A.3.1	Interfaces externas	47
A.3.2	Funcionales	48
A.3.3	No funcionales	50
A.3.4	Diseño e implementación	51
A.4	Verificación	51
B	Formulario sobre jugadores	53
B.1	Perfil del jugador	53
B.2	Uso de jueces en línea	56
B.3	Preferencias de recompensas y motivaciones	59
B.4	Información extra y retroalimentación	63
C	Casos de Prueba	65

List of figures

2.1	Proceso de uso de un juez en línea y ciclo de iteración	6
2.2	Captura de la tabla de clasificación global de Codeforces	8
2.3	Insignias disponibles en HackerRank	9
2.4	Estadísticas de Leetcode	9
2.5	Estadísticas de ¡Acepta el Reto!	10
4.1	Árbol de los requisitos de calidad para una aplicación web	20
4.2	Gráfico de la taxonomía de jugadores de Bartle. Se trata de una versión gráfica disponible en la página de Wikipedia correspondiente, basada en el dibujo taxonómico realizado por Bartle con ASCII en su artículo original.	23
4.3	Colores y topografía seleccionados para la guía de estilos	32

List of tables

2.1	Resumen de categorías de jueces en línea y sus elementos de gamificación	11
3.1	Comparativa de tecnologías frontend	16
3.2	Comparativa de frameworks backend	17
4.1	Logros propuestos	27
4.2	Niveles de dominio según puntos de experiencia (XP)	28
4.3	Acciones y recompensas en XP	30

Introducción

1.1 Motivación y antecedentes

¡Acepta el reto! vio la luz por primera vez en 2014 de la mano de los profesores Marco Antonio y Pedro Pablo Gómez Martín de la Universidad Complutense de Madrid. Como podemos leer en Pedro Pablo Gómez-Martín (2017) este juez online se creó como una herramienta educativa destinada a facilitar el aprendizaje de la programación, ofreciendo a los estudiantes un espacio en el que poner a prueba sus habilidades y conocimientos gracias a su amplio archivo de ejercicios en castellano. En los siguientes años siguió ganando popularidad, habiendo recibido ya más de un millón de envíos y contando con más de 5 mil usuarios¹. Su utilidad como juez online para ofrecer ejercicios interesantes en castellano ha provocado que su uso se haya extendido por centros educativos y haya servido de plataforma para concursos². Sin embargo, a pesar de su popularidad y su indiscutible valor pedagógico, *¡Acepta el reto!* presenta limitaciones significativas en cuanto a funcionalidades. La plataforma carece de herramientas avanzadas para la gestión de clases o concursos, y su diseño minimalista no permite a los usuarios compartir o visualizar el código de otros participantes, lo que limita las posibilidades de colaboración y aprendizaje entre pares.

Otra de las carencias más notables es la ausencia de un sistema que permita a los usuarios evaluar su progreso de manera tangible, ya que las estadísticas que proporciona a los usuarios son limitadas. Aunque la plataforma ofrece una amplia gama de ejercicios organizados por categorías, facilitando la búsqueda de temas específicos según los intereses o necesidades de cada usuario³ no incluye mecanismos para medir el avance individual, compararse con otros miembros de la comunidad o recibir reconocimientos por los logros alcanzados.

Esta situación representa una oportunidad única para mejorar y enriquecer el servicio que ofrece *¡Acepta el reto!*. Se puede observar que muchos de los jueces online comerciales, especialmente aquellos enfocados en la mejora de habilidades

¹Puedes encontrar más hitos importantes de *¡Acepta el reto!* aquí: <https://acceptaelreto.com/doc/history.php>

²<https://las12uvas.es/2025/#/quees>

³<https://acceptaelreto.com/problems/categories.php>

técnicas (ejemplos: HackerRank, LeetCode), han logrado un éxito notable en la retención de usuarios gracias a la implementación de sistemas de puntos, recompensas y medallas. También podemos comprobar que existen páginas de terceros que pretenden ofrecer un servicio de estadísticas⁴ para el juez, demostrando una demanda por parte de los usuarios, pero que al no ser una parte nativa de la aplicación no acceden directamente a la base de datos si no que realizan *scraping*⁵. Esto genera el riesgo de que los datos se queden desactualizados al cambiar la UI de la aplicación. Además, solo actualizan los datos una vez cada cierta cantidad de tiempo⁶ en lugar de actualizarse constantemente, por lo que a menudo están desactualizados y su utilidad para mostrar una imagen del progreso del usuario está limitada. *¡Acepta el reto!* también cuenta con algunas estadísticas de usuario que se muestran en la página de perfil, pero éstas están limitadas, no hay estadísticas para ejercicios, ni se hace notar un sistema consistente de gamificación a lo largo de la aplicación.

En este sentido, *¡Acepta el reto!* ya cuenta con una base sólida sobre la que integrar estas mejoras de manera nativa. La plataforma dispone de funcionalidades esenciales, como el seguimiento detallado de los envíos realizados por los usuarios, una categorización clara y accesible de los ejercicios, y una API robusta que permite el acceso a los datos de los usuarios de manera eficiente⁷. Estas características facilitan la implementación de un sistema de recompensas y visualización de progreso, que podría transformar la experiencia del usuario y elevar el valor educativo de la plataforma.

La motivación principal detrás de esta propuesta de mejora es, por tanto, añadir un valor significativo a la aplicación, reforzando su objetivo como espacio para la mejora de habilidades. La introducción de un sistema de medallas, junto con herramientas que permitan visualizar el progreso a lo largo del tiempo, no solo podría aumentar la motivación extrínseca de los usuarios, sino que también facilitaría la integración de la plataforma en entornos docentes (ejemplo: un profesor podría decidir otorgar un punto extra al alumno que obtenga más medallas durante un cierto periodo).

1.2 Objetivos

Como objetivos tenemos diseñar un panel de estadísticas y un sistema de logros para el juez online *¡Acepta el reto!*, que permita mostrar información relevante de la plataforma, y que mejore la experiencia del usuario, fomentando su participación mediante técnicas de gamificación. A continuación se listan las metas específicas que queremos alcanzar con este proyecto:

1. Identificar la información disponible en *¡Acepta el reto!* que pueda utilizarse para la elaboración de estadísticas
2. Establecer las métricas relevantes para representar la actividad de la plataforma.

⁴<https://aer.lluiscab.net/>

⁵Véase la cuarta definición para el término en el diccionario Cambridge: <https://dictionary.cambridge.org/dictionary/english/scraping>

⁶<https://aer.lluiscab.net/faq>

⁷<https://aceptaelreto.com/statistics/submissions.php>

3. Diseñar un panel de estadísticas que permita visualizar de forma clara la información del juez.
4. Mostrar estadísticas globales y de usuario que reflejen la actividad de la plataforma
5. Determinar qué información y logros resultan relevantes para los usuarios del juez online.
6. Diseñar un sistema de logros e insignias asociado a la actividad de los usuarios.
7. Mejorar la experiencia de usuario y la fidelización mediante elementos de gamificación.

1.3 Plan de trabajo

Para alcanzar los objetivos descritos en el apartado anterior, el desarrollo del proyecto se dividirá en distintas fases. Esto nos permitirá abordar el proyecto de una forma estructurada. Seguiremos las metodologías de diseño de software gamificado Morschheuser et al. (2018) que proponen un enfoque de diseño centrado en el usuario. En esta sección se describirá el proceso que seguiremos desde el análisis inicial del contexto hasta la implementación y evaluación de la solución final.

En la primera fase, se estudiará el funcionamiento del juez online ¡Acepta el reto!, así como la información y los datos de los que dispone la plataforma. El objetivo de esta fase es identificar aquellos elementos del juez que puedan ser utilizados para la obtención de estadísticas relevantes. También, se analizarán plataformas similares y otros jueces online para identificar buenas prácticas en aspectos como la presentación de estadísticas o la implementación de técnicas de gamificación. Este análisis permitirá tener una visión global del sistema, identificando sus capacidades y restricciones de cara al proyecto.

En esta segunda fase, a partir de la información identificada en la fase anterior, se establecerán las métricas que permiten reflejar la actividad del juez online, tanto a nivel global como a nivel de usuario. Estas métricas se seleccionarán teniendo en cuenta su relevancia para los usuarios y su utilidad para evaluar la actividad y evolución de la plataforma.

Durante la tercera fase, se diseñará el panel de estadísticas y los distintos dashboards que lo componen que permitirán mostrar tanto las estadísticas globales de la plataforma como las estadísticas individuales de los usuarios. Para ello, se estudiarán diferentes tipos de paneles y técnicas de visualización de la información, con el objetivo de seleccionar aquellas que permitan representar las métricas de forma clara y comprensible.

En la cuarta fase, se analizarán las necesidades, motivaciones y perfiles de los usuarios del juez online, apoyándose en un cuestionario, y de esa forma, diseñar un sistema de gamificación adaptado a esas necesidades. A partir de este análisis, y siguiendo los distintos principios de diseño de software gamificado, se definirán los diferentes elementos de gamificación ya sean puntos, niveles o insignias Zichermann y Cunningham (2011). Además, se diseñará el sistema de logros asociado a la actividad del usuario, para fomento su participación.

La quinta fase consistirá en implementar el panel de estadísticas y el sistema de gamificación diseñados en las fases anteriores. Tal y como se indica en libro mencionado anteriormente, la implementación se realizará siguiendo un enfoque iterativo, lo que permitirá realizar ajustes progresivos en función de los resultados obtenidos y de las pruebas realizadas.

Finalmente, en la sexta fase, se realizarán pruebas para verificar el correcto funcionamiento del sistema desarrollado y comprobar que se cumplen los objetivos planteados.

Estado del Arte de Jueces en Línea

2.1 Introducción

Un **juez en línea** (OJ, del inglés *Online Judge*) es un sistema automatizado accesible vía web que combina un repositorio de problemas de programación con un mecanismo de evaluación automática de soluciones. Los usuarios envían programas que representan propuestas de solución a alguno de los problemas planteados por el sistema. Cada problema define de manera precisa el formato de entrada, la salida esperada y una serie de restricciones que la solución debe cumplir. El juez en línea se encarga de compilar, ejecutar y probar las soluciones enviadas dentro de un entorno controlado y homogéneo, garantizando así condiciones de evaluación justas para todos los participantes, y la seguridad del propio sistema. Su objetivo principal es verificar tanto la corrección como la eficiencia del programa, generando posteriormente un veredicto ((Watanobe et al., 2022; Wasik et al., 2019)). Entre los veredictos más comunes se encuentran Accepted (AC), Wrong Answer (WA) o Time Limit Exceeded (TLE), aunque la nomenclatura y el conjunto exacto de resultados pueden variar ligeramente entre diferentes sistemas ¹.

Los primeros concursos de programación se remontan a la década de 1970 y fueron creciendo progresivamente en popularidad, lo que generó la necesidad de contar con sistemas capaces de evaluar automáticamente grandes volúmenes de código de forma rápida y objetiva. Esta necesidad se hizo especialmente evidente en el contexto de competiciones como la ICPC (International Collegiate Programming Contest), donde la corrección manual de soluciones resultaba inviable. En este escenario comenzaron a desarrollarse los primeros jueces en línea. Uno de los primeros OJs en alcanzar una popularidad global fue UVa Online Judge, creado por el profesor Miguel Ángel Revilla en la Universidad de Valladolid, y hecho disponible al público en general en 1997 (Revilla et al. (2008)). Desde entonces, estos sistemas han experimentado un crecimiento notable tanto en número de usuarios como en la cantidad de soluciones enviadas y problemas disponibles ².

¹Sobre las evaluaciones posibles en Uva Online Judge: https://onlinejudge.org/index.php?Itemid=31&id=16&option=com_content&task=view

²Estadísticas de uso en Uva Online Judge desde 1997: https://onlinejudge.org/index.php?option=com_onlinejudge&Itemid=23

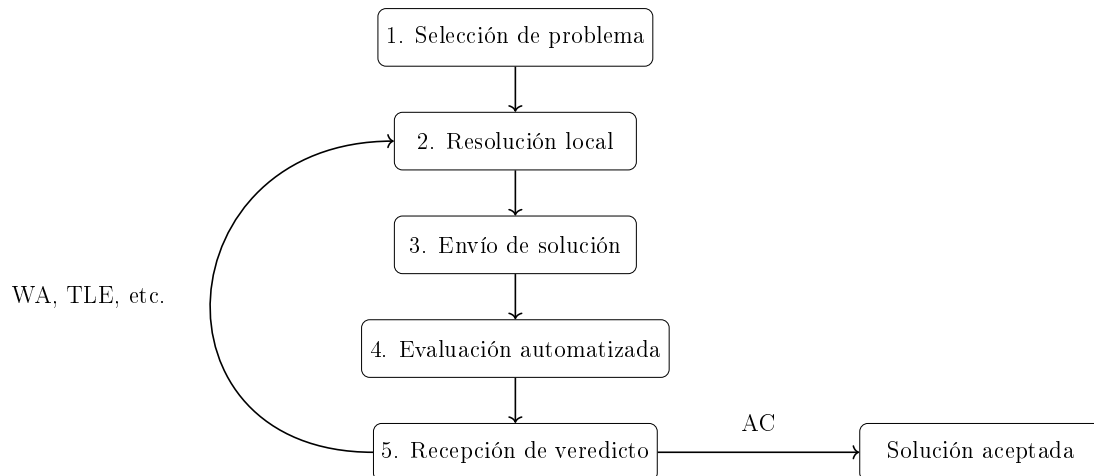


Figure 2.1: Proceso de uso de un juez en línea y ciclo de iteración

El uso de los jueces en línea se ha extendido además a múltiples ámbitos más allá de la programación competitiva. Ya en 1961, la Universidad de Stanford utilizaba un sistema automático para evaluar los programas en ALGOL desarrollados por sus estudiantes (Forsythe y Wirth (1965)). En la actualidad existen numerosos jueces en línea con un enfoque educativo, como CheckiO ³, Codecademy ⁴ o Jutge.org ⁵, que permiten a los usuarios aprender y practicar conceptos de programación y algoritmia tanto de forma individual como colectiva, por ejemplo en asignaturas universitarias o cursos especializados. Asimismo, los jueces en línea son ampliamente utilizados en procesos de reclutamiento laboral, donde se analizan las soluciones de los candidatos para evaluar sus habilidades técnicas, o como herramienta de preparación para entrevistas. Dos de los OJs más populares dentro de este ámbito son HackerRank, con más de 26,5 millones de desarrolladores registrados ⁶, y LeetCode ⁷.

2.2 Categorización de jueces de programación

Los jueces en línea pueden clasificarse en cuatro categorías principales en función del propósito para el que fueron diseñados. Esta clasificación permite comprender mejor las diferencias en sus objetivos, funcionalidades y público objetivo, así como el tipo de problemas y métricas de evaluación que emplean (Wasik et al. (2019)).

1. **Jueces de programación competitiva:** Los jueces de programación competitiva están orientados principalmente a la organización y desarrollo de competiciones de programación, tanto a nivel local como internacional. Estos sistemas suelen ofrecer rankings globales o regionales, sistemas de puntuación y límites estrictos de tiempo y memoria, ya que la eficiencia de las soluciones es un aspecto central. Los problemas están diseñados para poner a prueba

³<https://checkio.org/>

⁴<https://www.codecademy.com/>

⁵<https://jutge.org/>

⁶<https://www.hackerrank.com/about-us>

⁷<https://leetcode.com/>

la capacidad de los participantes en algoritmia, estructuras de datos y optimización, y el énfasis recae en comparar el rendimiento de los usuarios bajo condiciones homogéneas.

2. **Jueces educativos:** Los jueces educativos tienen como objetivo principal apoyar el aprendizaje de la programación. En este caso, el énfasis no se encuentra en la competición, sino en el proceso formativo. Estos sistemas suelen integrarse en contextos académicos o de autoaprendizaje, permitiendo a los usuarios practicar conceptos de programación de forma progresiva. La retroalimentación inmediata que ofrecen facilita la detección de errores y la mejora continua, siendo comunes en asignaturas universitarias, cursos especializados o plataformas de aprendizaje en línea.
3. **Jueces de reclutamiento:** Los jueces de reclutamiento están diseñados para evaluar las habilidades técnicas de candidatos en procesos de selección de personal. En este contexto, los problemas planteados buscan medir la capacidad del aspirante para resolver tareas concretas de programación dentro de un tiempo limitado. El OJ actúa como una herramienta objetiva que permite comparar soluciones y analizar el desempeño de los candidatos, además de servir como medio de preparación para entrevistas técnicas.
4. **Otros jueces:** Finalmente, existen otros jueces en línea orientados a nichos o dominios específicos. Estos sistemas se centran en áreas concretas de la programación o en tecnologías particulares, como puede ser la programación concurrente, el manejo de bases de datos o lenguajes específicos. Aunque su público suele ser más reducido, cumplen un papel importante al cubrir necesidades especializadas que no siempre están contempladas en los jueces generalistas.

2.3 Gamificación

Uno de los elementos de los jueces en línea modernos de especial interés para el tema de este trabajo es la gamificación.

La gamificación es una tendencia creciente en los jueces en línea, que busca motivar y fidelizar a los usuarios mediante la incorporación de elementos y mecánicas propias de los juegos (*games* en inglés). El proceso de gamificación no consiste en convertir la plataforma en un juego, sino en enriquecer la experiencia de usuario y aumentar su participación mediante el uso de mecánicas nuevas y la creación de una fuente de motivación extrínseca. Para que la gamificación sea eficaz, debe integrarse de forma coherente con los objetivos del sistema y el comportamiento deseado en los usuarios (Zichermann y Cunningham (2011)).

Los mecanismos de gamificación pueden variar según la categoría del juez, ya que cada tipo persigue objetivos diferentes:

1. **Jueces de programación competitiva:** En estas plataformas, la gamificación se centra en la comparación entre usuarios. Se utilizan tablas de

Rating: users participated in recent 6 months

	Who	#	=
1	Benq	165	3738
2	jiangly	188	3705
2	Kevin114514	99	3705
4	orzdevinwang	74	3670
5	tourist	290	3652
6	ksun48	307	3650
7	ecnerwala	209	3547
8	Radewoosh	260	3513
9	Nachia	70	3512
10	maroonrk	203	3443
11	dXqwq	83	3436
12	Um_nik	305	3427
13	Otomachi_Una	62	3413
14	cnfls_csy	58	3410
15	heuristica	68	3392
16	VivaciousAubergine	61	3386
17	qiuzx	77	3372
18	gyh20	76	3350
19	strapple	62	3321
20	BurnedChicken	137	3285

Figure 2.2: Captura de la tabla de clasificación global de Codeforces

clasificación y sistemas de puntuación, que permiten medir el progreso relativo de cada participante y fomentar la competencia. Por ejemplo, Codeforces implementa un sistema de rating basado en la participación en concursos, ayudando a los usuarios a identificar su posición dentro de la comunidad y a fijarse objetivos de mejora ⁸.

2. **Jueces educativos:** Estos sistemas aplican la gamificación para reforzar el aprendizaje individual y mantener la motivación del estudiante. Es habitual el uso de insignias (badges) y logros, que recompensan tanto la resolución de ejercicios como la constancia y el uso de diferentes lenguajes de programación. Plataformas como HackerRank emplean estas insignias visuales para ayudar al usuario a establecer metas y guiarle a alcanzarlas ⁹.
3. **Jueces de reclutamiento:** En este caso, la gamificación está dirigida tanto a demostrar habilidades y conocimientos ante potenciales empleadores como a la preparación para entrevistas técnicas. Aquí, las insignias y logros pueden actuar como una señal visible de competencia técnica. Además, debido al enfoque individualista tanto de los jueces de reclutamiento como de los educativos, las estadísticas se centran en mostrar el propio progreso y mejora del usuario.

⁸Sobre cómo Codeforces calcula la puntuación de usuario: <https://codeforces.com/blog/entry/102>

⁹<https://www.hackerrank.com/scoring/badges-and-medals>



Figure 2.3: Insignias disponibles en HackerRank

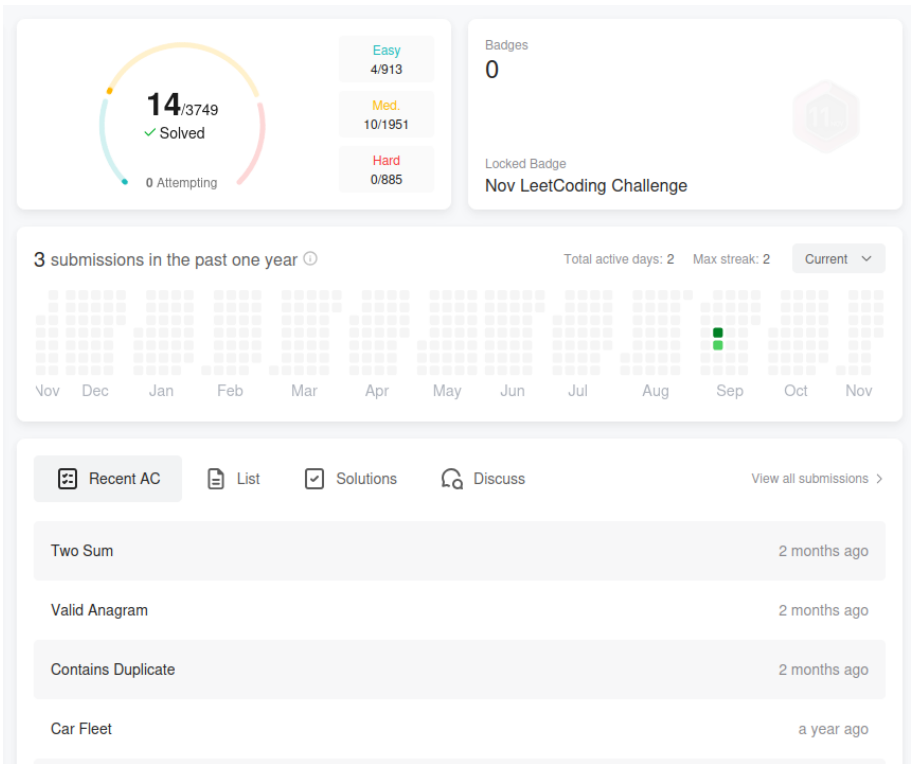


Figure 2.4: Estadísticas de Leetcode

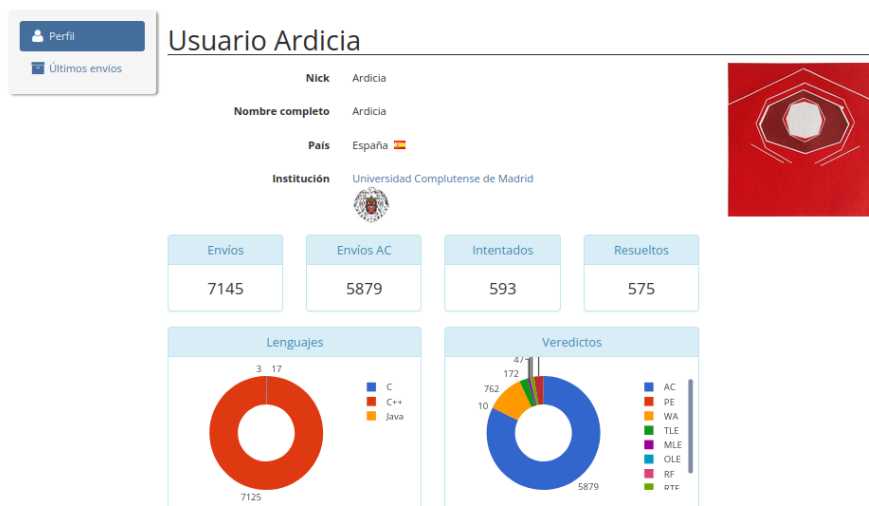


Figure 2.5: Estadísticas de ¡Acepta el Reto!

El OJ ¡Acepta el Reto! ya incluye varias herramientas de gamificación orientadas a seguimiento del progreso individual ¹⁰. En la pestaña de perfil se muestran estadísticas como:

- Cantidad de envíos realizados
- Cantidad de envíos correctos
- Cantidad de envíos intentados
- Cantidad de ejercicios resueltos
- Lenguajes más utilizados
- Veredictos más recibidos

Estas métricas permiten a los usuarios analizar su evolución y focalizar sus esfuerzos en mejorar áreas concretas, lo cual es coherente con su propósito educativo. Además, la plataforma cuenta con una pestaña de estadísticas que muestra las últimas submissions ¹¹.

2.4 Diseño y arquitectura

Aunque este trabajo no profundiza en la arquitectura interna de evaluación de los jueces en línea, resulta relevante comprender su estructura como aplicación web y los elementos que la distinguen de otros sistemas web convencionales.

A alto nivel, un juez en línea se compone de dos componentes principales. En primer lugar, se encuentra la parte web, que incluye la interfaz de usuario (frontend),

¹⁰<https://acceptaelreto.com/>

¹¹<https://acceptaelreto.com/statistics/submissions.php>

Categoría	Objetivo principal	Elementos de gamificación más comunes
Programación competitiva	Rendimiento	Puntuación, tablas de clasificación
Educativo	Aprendizaje	Insignias, logros y estadísticas de progreso
Reclutamiento	Mejorar habilidades	Insignias, logros y estadísticas de progreso

Table 2.1: Resumen de categorías de jueces en línea y sus elementos de gamificación

el servidor (backend) y la base de datos. Esta sección gestiona la interacción del usuario con la plataforma, permitiendo visualizar y seleccionar problemas, enviar código para su evaluación y consultar tanto estadísticas como resultados de sus envíos. La base de datos almacena la información necesaria para mantener un registro de los problemas disponibles, los envíos realizados y los datos de los usuarios.

En segundo lugar, y lo que realmente diferencia a un juez en línea de otras aplicaciones web, se encuentra el sistema de evaluación de código. Este subsistema permite recibir, ejecutar y evaluar código arbitrario enviado por los usuarios, una operación que en cualquier otra plataforma web constituiría un riesgo significativo de seguridad. Para ello, se utilizan demonios de evaluación que monitorizan continuamente los envíos y los ejecutan en entornos seguros y aislados, tales como contenedores, sandboxes o máquinas virtuales (Kuo et al. (2023)). La evaluación se realiza de manera asíncrona, de modo que el usuario puede continuar navegando por la plataforma mientras su envío es procesado. Una vez completada la evaluación, el veredicto se transmite a la parte web, que se encarga de mostrarlo al usuario.

Esta arquitectura permite separar de manera clara las funciones interactivas de la plataforma de las funciones de evaluación de código, garantizando tanto la usabilidad como la seguridad. La capacidad de ejecutar de forma segura código arbitrario constituye la preocupación central de estos sistemas y determina su diseño arquitectónico distintivo.

En síntesis, un juez en línea integra la infraestructura de una aplicación web tradicional con un subsistema especializado para la ejecución segura y asíncrona de código, lo que posibilita ofrecer una plataforma interactiva, confiable y segura para la resolución de problemas de programación.

2.5 Conclusiones

El análisis del estado del arte muestra que la visualización de estadísticas y la gamificación constituyen componentes fundamentales en los jueces en línea modernos, especialmente en aquellos con fines educativos y de reclutamiento. La presentación clara de métricas como envíos correctos, problemas resueltos, veredictos obtenidos y logros alcanzados permite a los usuarios evaluar su progreso, identificar áreas de mejora y mantener la motivación a lo largo del tiempo. Esta funcionalidad depende directamente de la arquitectura del juez, que separa la interfaz web de los subsis-

temas de evaluación de código, permitiendo actualizar los datos de manera segura y asíncrona. Por tanto, la creación de pantallas de estadísticas y logros no solo mejora la experiencia del usuario, sino que constituye un elemento esencial para el aprendizaje, la constancia y la participación activa en la plataforma.

Estado del Arte de Tecnologías para Desarrollo Web

3.1 Introducción

Las aplicaciones web modernas, en contraste con las páginas estáticas que las precedieron¹, se distinguen por su dinamismo, interactividad y capacidad para operar en tiempo real. Su arquitectura se sustenta en una estructura modular, compuesta por dos capas funcionalmente independientes: el frontend, encargado de la interacción con el usuario, y el backend, responsable de la lógica de negocio y el procesamiento de datos. Además, muchas aplicaciones incorporan una tercera capa dedicada al almacenamiento y gestión de datos, esencial para garantizar la persistencia y disponibilidad de la información relevante para su funcionamiento.

Para cumplir con las expectativas de los usuarios y las exigencias del mercado, las aplicaciones web modernas deben satisfacer una serie de requisitos fundamentales. Entre ellos, destacan el alto rendimiento, la escalabilidad y la mantenibilidad del código. Aunque la seguridad también puede ser una prioridad crítica dependiendo del contexto de la aplicación, estos tres primeros requisitos son universales en el desarrollo web actual.

En lo que respecta al rendimiento, las aplicaciones deben operar con rapidez y eficiencia. En 2016, la mitad de los usuarios de la web esperaban que una página se cargase en menos de dos segundos², lo que subraya la importancia de optimizar los tiempos de respuesta. Asimismo, la actualización constante y relevancia de los datos es crucial; de lo contrario, la aplicación podría perder utilidad y credibilidad.

Por otro lado, la escalabilidad y la mantenibilidad son cualidades que garantizan la longevidad y adaptabilidad del software. Una aplicación escalable puede manejar un crecimiento en la demanda sin degradar su rendimiento, mientras que una aplicación mantenible facilita su reutilización, integración con sistemas existentes y actualización continua a lo largo de su ciclo de vida, reduciendo costos y esfuerzo en el proceso.

¹En el siguiente artículo se puede leer más sobre el progreso histórico del desarrollo web: <https://www.geeksforgeeks.org/blogs/history-and-evolution-of-web-development/>

²<https://blog.google/products/admanager/the-need-for-mobile-speed/>

La arquitectura modular antes mencionada ofrece ventajas significativas para alcanzar estos objetivos. Por ejemplo, permite realizar modificaciones en una capa sin afectar a las demás. Esto significa que es posible actualizar una interfaz de usuario obsoleta en el frontend sin alterar la lógica de negocio subyacente en el backend, o incluso migrar a un sistema de base de datos más eficiente sin impactar la experiencia del usuario final.

A lo largo de los años, el desarrollo web se ha beneficiado de la aparición de frameworks y librerías especializadas, diseñadas para simplificar la implementación, mejorar el rendimiento y facilitar el mantenimiento de las aplicaciones. Estas herramientas suelen estar orientadas no solo a una capa específica de la aplicación, sino también a nichos tecnológicos o tipos de aplicaciones particulares, como plataformas de comercio electrónico, redes sociales o sistemas de gestión de contenido.

En los siguientes apartados, se analizarán las diferentes tecnologías disponibles para el desarrollo web, comparando sus especializaciones, puntos fuertes y posibles limitaciones, con el objetivo de ofrecer una visión crítica que permita seleccionar las herramientas más adecuadas según los requisitos del proyecto más adelante.

3.2 Frontend

La capa frontend desempeña un papel fundamental al ser la responsable de la interfaz de usuario y la interacción del cliente con el sistema. Su principal objetivo es garantizar una experiencia de usuario óptima, centrada en la usabilidad, la accesibilidad y la fluidez en la interacción. Esta capa se construye sobre tres tecnologías esenciales que operan de manera coordinada: HTML (HyperText Markup Language), que define la estructura del contenido; CSS (Cascading Style Sheets), que establece el diseño visual y la presentación; y JavaScript, que implementa la interactividad y el dinamismo de la interfaz.

La elección de las tecnologías frontend adecuadas es un aspecto crítico en el desarrollo de aplicaciones web, ya que influye directamente en el rendimiento, la escalabilidad y la mantenibilidad del proyecto. Según la encuesta de desarrolladores de Stack Overflow en 2024³, las tecnologías frontend más utilizadas incluyen React, jQuery, Angular y Vue.js. Sin embargo, la selección de una u otra depende de múltiples factores, como las características del proyecto, las habilidades del equipo de desarrollo y los requisitos de escalabilidad. En este sentido, el estudio comparativo realizado por Sekhar Emmanni (2023), destaca la importancia de evaluar estas tecnologías en función de sus ventajas técnicas y su alineación con los objetivos del proyecto.

React⁴, desarrollado por Meta, es una librería de JavaScript centrada en la construcción de interfaces de usuario mediante componentes reutilizables. Su enfoque en la modularidad y el uso de un DOM virtual permite optimizar el rendimiento, reduciendo los tiempos de carga y mejorando la eficiencia, como se evidencia en el estudio de Sekhar Emmanni (2023). Esto significa que es una framework ideal para las aplicaciones de alto rendimiento, o aquellas con un manejo de estados complejo.

³<https://survey.stackoverflow.co/2024/technology>

⁴<https://react.dev/>

Angular⁵, desarrollado y mantenido por Google, es un framework completo basado en el patrón MVC (Modelo-Vista-Controlador). Aunque su rendimiento puede ser ligeramente inferior en comparación con otras tecnologías modernas, su principal fortaleza radica en las herramientas integradas que ofrece para enrutamiento y gestión de formularios. Su sincronización entre la capa de datos y vista la avientaja en las aplicaciones donde la muestra de datos en tiempo real es necesaria. Se usa principalmente en aplicaciones empresariales a gran escala y aplicaciones de contenido dinámico.

Vue.js⁶ es un framework de JavaScript que destaca por su simplicidad y facilidad de integración en proyectos existentes. Estas características junto con su suave curva de aprendizaje, la han convertido en una herramienta ideal para el desarrollo de prototipados y extensiones a aplicaciones existentes.

Finalmente, JQuery⁷ es una librería para JavaScript que simplifica la manipulación de la página y el manejo de eventos. Aunque no es una framework a toda regla, sigue siendo muy popular para proyectos más pequeños y simples, o usada en conjunto con frameworks.

3.3 Backend

El backend representa la columna vertebral de las aplicaciones web modernas, asumiendo la responsabilidad de implementar la lógica de negocio, procesar datos y facilitar la comunicación con servicios externos. A diferencia del frontend, que se ejecuta en el navegador del usuario, el backend opera en el servidor, donde gestiona operaciones críticas como la autenticación de usuarios, la validación y procesamiento de datos, la interacción con bases de datos y la integración con APIs de terceros. El diseño y la eficiencia de esta capa son factores determinantes para asegurar el rendimiento óptimo, la escalabilidad y la seguridad de las aplicaciones.

De acuerdo con los resultados de la encuesta de desarrolladores de Stack Overflow anteriormente mencionada, las tecnologías backend más utilizadas en la actualidad incluyen Node.js, Express, Flask y Django. Cada una de estas tecnologías presenta características distintivas que las hacen adecuadas para diferentes tipos de proyectos. Además, como se detalla en el estudio comparativo de Frameworks populares para backend realizado por Buljic et al. (2025), estos frameworks se basan en lenguajes de programación y tecnologías distintas, lo que influye en su rendimiento (medido en términos de latencia y throughput) y, por tanto, en su idoneidad para distintos escenarios de aplicación.

Node.js⁸ es un entorno de ejecución de JavaScript en el lado del servidor, diseñado específicamente para el desarrollo de aplicaciones escalables y de alto rendimiento. Como se muestra en los resultados del estudio comparativo realizado por Lei et al. (2014), su arquitectura basada en eventos y su modelo de entrada/salida (E/S) no bloqueante lo convierten en una opción ideal para aplicaciones que requieren manejar

⁵<https://angular.dev/>

⁶<https://vuejs.org/>

⁷<https://jquery.com/>

⁸<https://nodejs.org/es>

Tecnología	Tipo	Características	Uso típico
React	Librería	Componentes reutilizables, DOM virtual, modularidad, alto rendimiento	Aplicaciones de alto rendimiento, manejo de estados complejos
Angular	Framework completo	Basado en MVC, herramientas integradas para enrutamiento y gestión de formularios, sincronización de datos en tiempo real	Aplicaciones empresariales a gran escala, contenido dinámico
Vue.js u	Framework	Simplicidad, facilidad de integración, suave curva de aprendizaje	Prototipado, extensiones a aplicaciones existentes
jQuery	Librería	Simplificación de manipulación de página y manejo de eventos	Proyectos pequeños y simples, uso conjunto con frameworks

Table 3.1: Comparativa de tecnologías frontend

múltiples conexiones simultáneas, como sistemas en tiempo real o APIs. Además, Node.js cuenta con un ecosistema robusto, gracias a npm (Node Package Manager), que proporciona una amplia variedad de módulos para extender funcionalidades y agilizar el proceso de desarrollo.

Por su parte, Express⁹ es un framework minimalista para Node.js, diseñado para simplificar la creación de servidores web y APIs. Le permite a los desarrolladores construir aplicaciones robustas y eficientes con una configuración mínima, ofreciendo la flexibilidad necesaria para adaptarse a las necesidades específicas de cada proyecto.

Aunque Next.js¹⁰ es reconocido principalmente como un framework de React para el desarrollo de interfaces de usuario, su capacidad para implementar Server-Side Rendering (SSR) y rutas API lo posiciona como una herramienta híbrida que también puede gestionar lógica de backend. Esta característica permite a los desarrolladores construir aplicaciones full-stack utilizando un único framework, lo que

⁹<https://expressjs.com/es/>

¹⁰<https://nextjs.org/>

simplifica la arquitectura del proyecto y reduce su complejidad.

En el ecosistema de Python, tanto Django¹¹ como Flask¹² son frameworks ampliamente utilizados para el desarrollo de aplicaciones web. Sin embargo, presentan diferencias significativas en cuanto a su enfoque y funcionalidades. Django es un framework robusto y completo, que incluye una amplia gama de herramientas integradas, lo que lo hace especialmente adecuado para el desarrollo de sistemas empresariales complejos. En contraste, Flask se caracteriza por su ligereza y flexibilidad, siendo una opción preferida para proyectos más pequeños o aquellos que requieren una mayor personalización.

Frame-work	Lenguaje	Tipo	Características	Uso típico
Node.js	JavaScript	Entorno de ejecución	Arquitectura basada en eventos, E/S no bloqueante, escalable, ecosistema npm	Aplicaciones en tiempo real, APIs
Express	JavaScript	Framework minimalista	Minimalista, fácil configuración, ideal para APIs y servidores web	APIs RESTful, aplicaciones web ligeras
Next.js	JavaScript / Type-Script	Framework full-stack	SSR, rutas API, integración con React	Aplicaciones full-stack, SSR
Django	Python	Framework completo	Baterías incluidas, ORM, admin panel, seguridad integrada	Sistemas empresariales
Flask	Python	Micro-framework	Ligero, flexible, fácil de extender	Proyectos pequeños, APIs

Table 3.2: Comparativa de frameworks backend

3.4 Conclusión

El cambio de paradigma en el desarrollo web actual está impulsado por la necesidad de crear aplicaciones más rápidas, interactivas y eficientes. En este contexto, surge una clara necesidad de adoptar tecnologías que no solo agilicen la creación de aplicaciones, sino que también permitan una fácil reutilización, integración y escalabilidad a largo plazo.

Por ello, los frameworks y librerías modernos han demostrado ser herramientas esenciales para los desarrolladores. Permiten construir aplicaciones con una gran funcionalidad de manera rápida y eficiente, proporcionando herramientas listas para usar que reducen considerablemente el tiempo de desarrollo. Además, están altamente especializados, lo que significa que existen opciones adaptadas a prácticamente todas las necesidades, y nos basta con conocer nuestras opciones, nuestra

¹¹<https://www.djangoproject.com/>

¹²<https://flask.palletsprojects.com/en/stable/>

aplicación y nuestro entorno de desarrollo para elegir una solución potente y adaptada.

Chapter 4

Diseño de la Aplicación

En esencia, hay dos enfoques fundamentales del diseño: el ideal artístico de expresarse a sí mismo y el ideal de la ingeniería de resolver un problema para un cliente.

Jakob Nielsen

4.1 Introducción

El análisis, la planificación y el diseño de un proyecto software constituyen etapas fundamentales dentro de su ciclo de desarrollo (Pressman (2010)). Un enfoque basado en un análisis exhaustivo de los requisitos, junto con un diseño centrado en las necesidades del usuario, no solo permite garantizar el cumplimiento de los objetivos funcionales del sistema, sino que también contribuye a reducir la aparición de errores, optimizar los costes de desarrollo y facilitar tanto el mantenimiento como las futuras actualizaciones de la aplicación.

En este contexto, el diseño puede entenderse como un factor clave para la incorporación de calidad al producto software. Tal y como señalan Olsina et al. (2001), la calidad de una aplicación no es un atributo único, sino el resultado de la combinación de múltiples características. Estos autores proponen un *árbol de requerimientos de calidad* que identifica los principales elementos sobre los que es necesario incidir a lo largo del desarrollo del proyecto, especialmente en el caso de aplicaciones web, donde aspectos como la usabilidad, la eficiencia o la confiabilidad adquieren un papel especialmente relevante.

Atendiendo a estos principios, el diseño de la aplicación se aborda en este capítulo desde una perspectiva multidimensional. En primer lugar, se definen las funcionalidades del sistema a partir de un análisis detallado de los usuarios finales y de los objetivos del proyecto, seleccionando aquellas capacidades que resultan esenciales para satisfacer los requisitos previamente identificados. De forma paralela, se plantea el diseño de la arquitectura técnica del sistema, describiendo sus distintos componentes y justificando la selección de las tecnologías empleadas para su implementación, siempre teniendo en cuenta criterios de eficiencia, confiabilidad y

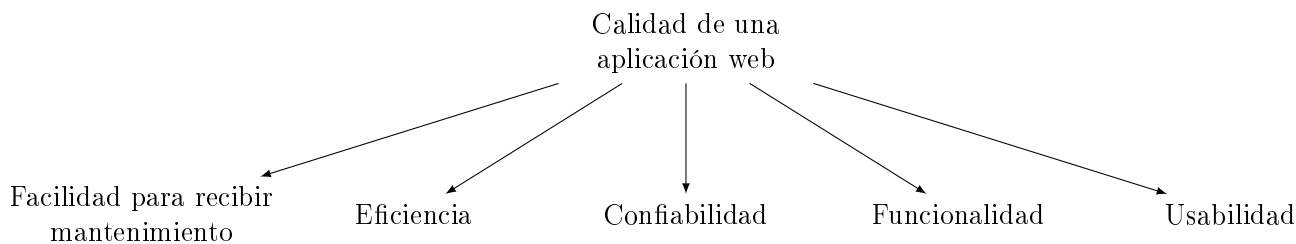


Figure 4.1: Árbol de los requisitos de calidad para una aplicación web

facilidad de mantenimiento.

Finalmente, se desarrolla el diseño de la interfaz y de la experiencia de usuario (UI/UX), prestando especial atención a la forma en que se presentan las estadísticas, a los elementos gráficos utilizados y a la estructura general de la aplicación desde el punto de vista del usuario. En esta fase, la usabilidad se convierte en el eje central del diseño, con el objetivo de garantizar una interacción intuitiva, clara y satisfactoria con el sistema.

4.2 Análisis de requisitos

El primer paso de nuestro diseño será identificar los requisitos que el software debe suplir. Este análisis sirve de base para todo el diseño, ya que establece qué se debe implementar, con qué eficiencia, y quién es nuestro público objetivo.

A partir del análisis de requisitos recogido en el Apéndice A, se definen en esta sección las funcionalidades que conforman la aplicación. Dichas funcionalidades responden directamente a los requisitos funcionales identificados en la SRS y constituyen el núcleo del sistema desde el punto de vista del usuario final.

El diseño funcional se ha orientado a proporcionar información relevante, clara y actualizada sobre la actividad de los usuarios en el juez *¡Acepta el reto!*, así como a introducir mecanismos de motivación adicionales mediante la gamificación. Para ello, las funcionalidades se agrupan en cuatro bloques principales: estadísticas de usuario, estadísticas de ejercicios, sistema de logros y tabla de clasificación.

4.2.1 Estadísticas de usuario

Una de las funcionalidades centrales del sistema es la visualización de estadísticas asociadas a cada usuario (REQ-FUN-001). Estas estadísticas permiten reflejar de manera cuantitativa el progreso y la actividad del usuario dentro del juez online.

Entre la información mostrada se incluyen, entre otros indicadores, el número total de ejercicios resueltos, el número de envíos realizados y la tasa de aciertos. Estos datos se presentan de forma agregada y comprensible, con el objetivo de que el usuario pueda evaluar su evolución a lo largo del tiempo y detectar posibles áreas de mejora.

Las estadísticas de usuario se actualizan automáticamente cuando se producen eventos relevantes en el sistema, como la evaluación de un nuevo envío (REQ-FUN-003). De este modo, se garantiza que la información mostrada sea coherente con el

estado real del sistema y no requiera ninguna acción manual por parte del usuario.

4.2.2 Estadísticas de ejercicios

Además de las estadísticas individuales, el sistema proporciona estadísticas asociadas a los ejercicios disponibles en el juez (REQ-FUN-002). Estas estadísticas se calculan de forma agregada a partir de la actividad de todos los usuarios y permiten ofrecer una visión global de cada problema.

Entre los indicadores considerados se encuentran el número de intentos realizados y el porcentaje de éxito. Esta información resulta útil para que los usuarios puedan estimar la dificultad relativa de los ejercicios, así como su popularidad dentro de la plataforma.

Al igual que ocurre con las estadísticas de usuario, las estadísticas de ejercicios se actualizan automáticamente conforme se registran nuevos envíos evaluados, garantizando la consistencia y fiabilidad de los datos mostrados.

4.2.3 Sistema de logros

Con el objetivo de incrementar la motivación y fomentar la participación activa, el sistema incorpora un sistema de logros o medallas (REQ-FUN-004). Los logros se otorgan automáticamente cuando un usuario cumple una serie de condiciones predefinidas, como alcanzar un determinado número de ejercicios resueltos.

Cada logro representa un hito alcanzado por el usuario y actúa como una recompensa simbólica que refuerza comportamientos positivos dentro del sistema. El diseño funcional del sistema de logros debe ser extensible, permitiendo la incorporación de nuevos logros en el futuro sin necesidad de modificar la lógica principal de la aplicación.

Los usuarios pueden consultar en todo momento la lista de logros obtenidos (REQ-FUN-005), accediendo a una vista específica en la que se muestran de manera clara y ordenada. Esta funcionalidad contribuye a reforzar la percepción de progreso y a mantener el interés del usuario a largo plazo.

4.2.4 Tabla de clasificación

Otra funcionalidad clave del sistema es la tabla de clasificación de usuarios (REQ-FUN-006 y REQ-FUN-007). Esta tabla ordena a los usuarios en función de su puntuación, calculada a partir de su actividad y de los logros alcanzados.

La tabla de clasificación se actualiza automáticamente cada vez que se produce un evento que afecta a la puntuación de un usuario, garantizando que la información mostrada sea siempre actual. Desde el punto de vista funcional, se permite a los usuarios consultar tanto su propia posición como la del resto de participantes, favoreciendo la comparación y la competencia.

4.2.5 Compartición de estadísticas y logros

Finalmente, el sistema contempla la posibilidad de compartir estadísticas o logros obtenidos por un usuario (REQ-FUN-008). Esta funcionalidad permite generar una imagen o enlace que represente visualmente un logro o un conjunto de estadísticas, facilitando su difusión fuera de la plataforma.

Desde el punto de vista funcional, esta característica busca aumentar la visibilidad del progreso del usuario y reforzar la motivación mediante el reconocimiento externo.

4.3 Diseño del sistema de gamificación

Como se ha establecido previamente, el objetivo de este proyecto es incorporar componentes de estadísticas y logros en el juez online *¡Acepta el reto!* para impulsar la motivación, la fidelización de los usuarios y facilitar su proceso de aprendizaje. Los elementos que se pretenden integrar se corresponden con mecánicas de juego. Por consiguiente, el proceso de diseño de estas funciones en la aplicación no consiste únicamente en añadir características aisladas, sino en aplicar de manera consciente y estructurada principios de diseño de juegos relevantes para generar experiencias que resulten atractivas y significativas para los usuarios. Este proceso se reconoce como **gamificación** (del inglés *game*, juego).

4.3.1 Introducción a la gamificación

El concepto de la gamificación ha emergido a partir de comienzos del siglo XXI como un campo interdisciplinario que combina principios de diseño de juegos, psicología motivacional y análisis de comportamiento humano para enriquecer la interacción con sistemas digitales¹. En términos generales, la gamificación se define como la utilización de elementos y mecánicas de juego con el propósito de motivar a los usuarios, aumentar su participación y ayudarles a resolver problemas (Zichermann y Cunningham (2011)). En este sentido, la gamificación representa una herramienta para fomentar comportamientos deseables y no una solución mágica o independiente de contexto. Por ello, es crucial evitar prácticas de “gamificación por gamificar” que carezcan de un análisis previo sobre su pertinencia y efectos esperados. También conviene considerar la diferencia entre la motivación extrínseca (que procede de estímulos externos) e intrínseca (que procede de un deseo interno) a la hora de gamificar. Nuestro objetivo será producir una fuente de motivación extrínseca que ofrezca resultados predecibles en los usuarios, pero sea tan agradable y fluida para el usuario que parezca intrínseca.

De forma complementaria, otros autores la describen como el enriquecimiento de software mediante características de diseño propias de los juegos, con el objetivo de generar experiencias tan atractivas y absorbentes como las que ofrecen los

¹Se recomienda leer la siguiente presentación realizada por Amy Jo Kim en 2008, en la que se muestra claramente el panorama de la gamificación en la década de los 2000: <https://es.slideshare.net/slideshow/putting-the-fun-in-functiona/300487#1>

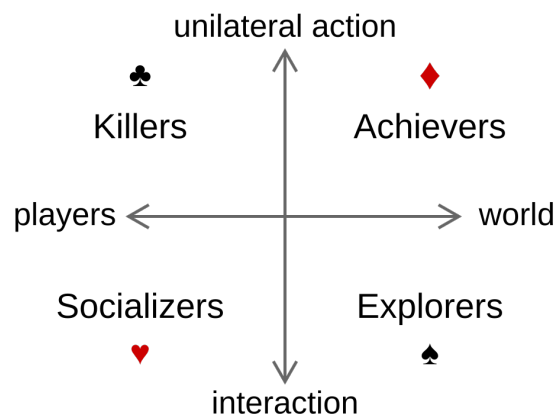


Figure 4.2: Gráfico de la taxonomía de jugadores de Bartle. Se trata de una versión gráfica disponible en la página de Wikipedia correspondiente, basada en el dibujo taxonómico realizado por Bartle con ASCII en su artículo original.

juegos tradicionales (Morschheuser et al. (2018)). Este enfoque subraya que el diseño gamificado va más allá de la simple aplicación de recompensas superficiales y requiere integrar conocimientos sobre motivación y diseño de juegos para que las interacciones resulten realmente significativas y no solo un añadido cosmético.

Por lo tanto, se debe realizar un análisis de los usuarios (en esta sección también llamados *jugadores*) y mecánicas, para determinar qué es más conveniente en el contexto dado.

4.3.2 Análisis de jugadores y mecánicas

En la sección anterior hemos destacado la importancia de la motivación extrínseca para crear un buen sistema de gamificación. Sin embargo, todo diseño de gamificación debe realizarse con el foco en el jugador. Comprender qué motiva a los jugadores es esencial para construir un sistema gamificado exitoso.

Según Bartle (1996), los jugadores pueden clasificarse en cuatro tipos no excluyentes, que determinan como debe diseñarse la experiencia de juego para poder fidelizarlos. Las cuatro categorías son:

- Exploradores: Buscan descubrir y experimentar.
- Logradores (Achievers): Se enfocan en completar objetivos y acumular recompensas.
- Socializadores: Priorizan la interacción con otros jugadores.
- Competidores (Killers): Disfrutan superando a otros, incluso a costa de la colaboración. Es decir, no solo buscan ganar, si no que además buscan hacer perder a otro.

Para determinar a qué categoría pertenecen nuestros jugadores objetivo en las distintas áreas de la aplicación, se diseñó un formulario cuyos resultados se recogen en el apéndice B. El análisis de estos datos revela patrones relevantes en relación

con las preferencias y motivaciones de los jugadores. A continuación, se presentan las principales implicaciones de dichos resultados para cada una de las categorías del modelo de Bartle:

1. Competidores (Killers): Tal y como cabía esperar en un entorno de programación competitiva, los resultados reflejan una presencia notable de comportamientos asociados al perfil competitivo, especialmente en contextos de comparación directa y resolución de problemas de alta dificultad. El fuerte interés mostrado por las tablas de clasificación (con más del 90% de los encuestados manifestando su deseo de aparecer en ellas), así como la preferencia por mecanismos de filtrado por nivel o tipo de problema, refuerzan esta interpretación. No obstante, este perfil aparece de forma más limitada en aspectos relacionados con la satisfacción personal, la motivación intrínseca o el reconocimiento individual. En consecuencia, aunque resulta imprescindible incorporar mecánicas competitivas como rankings y comparaciones entre usuarios, estas no deben dominar el diseño global del sistema de gamificación.
2. Logradores (Achievers): El perfil de logrador es el más consistente a lo largo de los distintos contextos analizados y adquiere especial relevancia en relación con la satisfacción y el progreso personal. Este comportamiento está alineado con el carácter educativo de la plataforma y con los resultados de la encuesta, donde los usuarios valoran especialmente poder visualizar su mejora a lo largo del tiempo, el número de envíos correctos o el tiempo invertido. Desde el punto de vista del diseño, esto justifica la implementación de un sistema variado de insignias y logros, así como estadísticas claras y detalladas que permitan a los usuarios seguir su evolución, comparar su rendimiento entre distintos periodos o lenguajes de programación y recibir retroalimentación constante sobre su progreso.
3. Exploradores (Explorers): El perfil explorador destaca principalmente en los contextos de aprendizaje. Los usuarios muestran interés por descubrir nuevas estrategias, experimentar con distintos tipos de problemas o lenguajes, y obtener reconocimiento por enfoques menos convencionales. Este comportamiento se refleja, por ejemplo, en el interés por logros asociados a tipos de problemas, características específicas de los envíos o lenguajes utilizados. En consecuencia, el diseño del sistema de gamificación debe contemplar la inclusión de elementos que fomenten la exploración, como contenido desbloqueable o insignias especiales, incluyendo un número limitado de *insignias sorpresa* o *medallas misteriosas*, cuya aceptación, aunque no unánime, resulta significativa para una parte de los usuarios.
4. Socializadores (Socializers): Aunque se trata del perfil menos predominante en términos generales, los socializadores aparecen en aquellos contextos en los que la colaboración contribuye al aprendizaje. Estos jugadores encuentran satisfacción en cooperar para alcanzar soluciones o en enseñar a otros. Dado que *¡Acepta el reto!* no dispone de funcionalidades sociales como foros o soporte para grupos, y que el desarrollo de dichas características queda fuera del alcance de este proyecto, unido a la baja representación de este perfil, no

se implementarán técnicas de gamificación específicas para este grupo, como sistemas de puntos por participación en foros.

A partir del análisis realizado, se concluye que los usuarios de la plataforma no responden a un único perfil de jugador, sino que alternan entre distintos roles en función del contexto, predominando claramente los perfiles de logrador, competidor y explorador. En general, los jugadores muestran una preferencia marcada por mecánicas que refuercen el progreso individual, la comparación objetiva del rendimiento y el dominio del “mundo del juego”, entendido como el conjunto de habilidades y conocimientos propios de la programación competitiva, frente a la interacción social con otros usuarios.

Estas conclusiones justifican un diseño de gamificación centrado en estadísticas detalladas, retroalimentación continua, sistemas de logros significativos y tablas de clasificación segmentadas por nivel. De este modo, el sistema propuesto se alinea tanto con las motivaciones detectadas en la encuesta como con los objetivos formativos de la plataforma.

4.3.3 Propuesta de diseño

Con toda la información relevante sobre los jugadores objetivo y las mecánicas disponibles recabada, se realiza una propuesta de diseño de sistema de gamificación basada en el marco teórico MDA (*Mecánicas, Dinámicas, Estética*), presentado en Zichermann y Cunningham (2011). Esto quiere decir que la propuesta está dividida en las mecánicas a introducir, las dinámicas deseadas de los jugadores con estas mecánicas, y el sentimiento deseado en los jugadores al interactuar con las mecánicas.

4.3.3.1 Mecánicas

El sistema de logros es la primera de las mecánicas ideada. Para su diseño, se establecieron las siguientes directrices que se corresponden con las necesidades de los usuarios establecidas en la sección anterior:

- Simplicidad y claridad: Cada logro debe poder describirse en una o dos frases como máximo, lo que garantiza su fácil comprensión por parte de los usuarios.
- Permanencia: Los logros deben corresponder a acciones irreversibles o que no puedan perderse. Una vez obtenido, el usuario lo conservará de forma definitiva. Por ejemplo, no se contempla un logro por conseguir todos los logros disponibles, ya que su validez se vería comprometida al incorporarse nuevos logros en el futuro.
- Categorización por dificultad: Los logros se clasifican en tres niveles (Bronce, Plata y Oro) según su dificultad o relevancia. Esta jerarquía, inspirada en el sistema de medallas del ámbito deportivo, resulta intuitiva y reconocible para los usuarios, y sirve como indicador de progreso. Además, otros jueces online, como HackerRank, ya implementan esta jerarquía de logros.

- Categorización por tipo de acción: Los logros también se agrupan según el aspecto que premian, como la calidad de los envíos o el uso de lenguajes de programación. Las categorías definidas son:
 - Onboarding: Ayudan a los usuarios a aclimatarse con el sistema de gamificación.
 - Número de problemas: Premian al usuario por haber resuelto una cierta cantidad de problemas.
 - Uso de lenguajes: Premian al usuario por haber usado ciertos lenguajes.
 - Rachas: Premian al usuario por su consistencia en diferentes aspectos.
 - Calidad: Premian al usuario por ideas soluciones buenas.
 - Categorías: Premian al usuario por hacer ejercicios de determinadas categorías. Las categorías que contemplamos son: construcciones de programación, estructuras de datos, algoritmia, matemáticas, grafos y geometría.
- Logros sorpresa: Con el objetivo de fomentar la exploración y la sensación de descubrimiento en los usuarios, especialmente en aquellos con perfiles de *logrador* o *explorador*, se han incluido dos logros cuya descripción no es accesible hasta que se obtienen. Estos logros no ofrecen pistas sobre cómo conseguirlos y pueden desbloquearse en cualquier momento.

En la tabla 4.1 se detallan los logros propuestos a partir de las directrices definidas, junto con su categorización:

La distribución final de los logros queda establecida en 5 de nivel *Bronce*, 6 de nivel *Plata* y 7 de nivel *Oro*. Esta proporción garantiza un equilibrio en la progresión del usuario, al tiempo que diversifica las categorías de logros según su tipo. Asimismo, se han incluido solo dos logros sorpresa, una decisión que busca satisfacer, en la mayor medida posible, las preferencias de los usuarios encuestados, cuyas opiniones sobre este aspecto estuvieron divididas.

La siguiente mecánica a definir es el tema de las estadísticas. Dispondremos tanto de estadísticas de cada usuario, como de cada ejercicio.

Para los usuarios se han ideado las siguientes estadísticas, considerandolas informativas para entender rápidamente el nivel de experiencia y conocimiento de un determinado usuario, o para servir de referencia a la hora de intentar obtener ciertos logros:

1. Número de problemas resueltos: Total acumulado de ejercicios completados con éxito.
2. Posición en la tabla de clasificación global: Ubicación del usuario en el ranking general de la plataforma, basada en sus puntos de experiencia.
3. Proporción de veredictos obtenidos en envíos.
4. Evolución temporal de envíos realizados:

Descripción	Nivel	Categoría
Creación de una cuenta	Bronce	Onboarding
Realización del primer envío	Bronce	Onboarding
Obtención de 5 logros	Plata	Onboarding
Resolución de 10 problemas	Bronce	Número de problemas
Resolución de 50 problemas	Plata	Número de problemas
Resolución de 100 problemas	Plata	Número de problemas
Resolución de 500 problemas	Oro	Número de problemas
Resolución de 25 problemas en C	Oro	Uso de lenguajes
Resolución de 25 problemas en C++	Oro	Uso de lenguajes
Resolución de 25 problemas en Java	Oro	Uso de lenguajes
Haber realizado envíos con 3 lenguajes diferentes	Plata	Uso de lenguajes
Consecución de una racha de 5 envíos aceptados a la primera	Oro	Rachas
Realización de envíos en 7 días consecutivos	Bronce	Rachas
Resolución de un problema en el primer intento	Bronce	Calidad
Envío correcto dentro del 25% de soluciones más rápidas para un problema	Plata	Calidad
Resolución de un problema de cada categoría	Oro	Categorías
Envío correcto que iguale o mejore el tiempo de ejecución récord para un problema	Oro	Calidad (sorpresa)
Realización de envíos en cada franja horaria	Plata	Rachas (sorpresa)

Table 4.1: Logros propuestos

- Visualización de los envíos realizados cada día.
 - Racha actual de envíos: Número actual de días consecutivos con al menos un envío.
 - Racha máxima: Mayor número de días consecutivos con envíos registrados en el historial del usuario.
5. Puntos de experiencia (XP) y nivel de dominio a lo largo del tiempo.
 6. Proporción de lenguajes de programación utilizados.

Se eliminarían además las estadísticas actualmente existentes de número de problemas intentados, número de envíos totales realizados, y número de envíos AC. En el caso de los dos primeros debido a que no responden a ninguna necesidad específica de un estudiante que quiere conocer su estado en el sistema, sobretodo en su representación numérica actual. Serían remplazadas por las estadísticas del punto 4, que además de dar la información sobre cantidad de envíos realizados lo representan y comparan en el tiempo, aplicandolo a una necesidad real (entender el progreso en el tiempo). Por otro lado el número de envíos AC es redundante cuando ya existe

una estadística que indica el número de problemas resueltos y otra que indica la proporción de veredictos obtenidos.

El sistema de puntos implementado se basa en el modelo de puntos de experiencia (XP), el cual se diferencia de otros sistemas de puntos por cumplir tres condiciones fundamentales:

- No monetizable: Los puntos de experiencia no pueden canjearse por recompensas ni utilizarse como moneda dentro del sistema. Su función es exclusivamente indicativa del progreso del usuario.
- Acumulación por actividad: Los usuarios obtienen XP por todas las acciones relevantes realizadas en la plataforma, específicamente por:
 - Envíos de soluciones.
 - Desbloqueo de logros.
- Incremento permanente: Los puntos de XP solo pueden aumentar; nunca decrecen, lo que garantiza un progreso constante y sin penalizaciones.

Para facilitar la visualización del progreso, se introduce el concepto de **niveles de dominio**, que asignan un título al usuario según el rango de XP acumulado. Estos niveles permiten una comprensión inmediata de la posición del usuario dentro del sistema y fomentan la motivación mediante metas claras.

Los niveles de dominio establecidos se detallan en la Tabla 4.2.

Nivel de dominio	Rango de XP
Aprendiz	0 - 100 XP
Competente	101 - 500 XP
Hábil	501 - 1,000 XP
Especialista	1,001 - 2,000 XP
Maestro	Más de 2,000 XP

Table 4.2: Niveles de dominio según puntos de experiencia (XP)

Está planteado de tal manera que sea relativamente fácil abandonar los primeros niveles, pero la progresión sea más compleja con el tiempo. Por ejemplo, para pasar de aprendiz a competente se necesitan solo 100 puntos, mientras que para pasar de especialista a maestro se necesitan 1000. El sistema de XP permite generar una tabla de clasificación con dos vistas complementarias:

- Clasificación global: Muestra la posición del usuario en relación con todos los participantes del sistema.
- Clasificación por nivel: Permite comparar el desempeño del usuario con otros usuarios que compartan su mismo nivel de dominio.

Este diseño fomenta la interacción competitiva entre los usuarios, al tiempo que ofrece una recompensa tangible por interactuar con el sistema. Además, al vincularse

directamente con el desbloqueo de logros, el sistema refuerza la sensación de progreso y logro personal.

Una característica importante a tener en cuenta tanto en el diseño como la evaluación de un sistema de gamificación es el problema de **la superación del jefe final**. Este problema se refiere a cuando un sistema de gamificación, sobretodo uno que se introduce en un sistema ya existente con años de experiencia como es *¡Acepta el reto!*, se vuelve redundante porque un número considerable de usuarios obtienen el máximo nivel de dominio, todos los logros, etc. Se pierde la fuente de motivación extrínseca y se inutiliza el esfuerzo invertido en el sistema. Para evitar esto, todas las mecánicas del sistema de gamificación deben ser fácilmente extensibles. En nuestro caso, las estadísticas no cuentan con un “nivel máximo” y por lo tanto no corren riesgo de volverse obsoletas, ya que su único objetivo es mostrarle su situación al usuario, no evaluarle o recompensarle por ésta. Por otro lado, el sistema de logros es fácilmente extensible añadiendo nuevos logros que cumplan las condiciones de diseño anteriormente definidas. Por ejemplo, podrían añadirse nuevos logros que recompensen la cantidad de ejercicios resueltos según se añadan más ejercicios a la plataforma. Finalmente, los niveles de dominio del sistema de puntos podrían ser reasignados a cantidades de XP diferentes, o definirse un nuevo nivel con su rango de XP correspondiente.

4.3.3.2 Dinámicas

En esta sección profundizamos en una de las categorías mencionadas previamente, pero no desarrollada: la bienvenida (*onboarding*). Esta etapa se centra en técnicas diseñadas para guiar al usuario en su primer contacto con el sistema, evitando la sobrecarga de información y previniendo la sensación de desorientación. Los logros resultan una herramienta especialmente efectiva para este propósito, ya que actúan como indicadores interactivos que orientan al usuario sobre las acciones más relevantes y sus beneficios. Por ejemplo, el logro otorgado al *crear una cuenta* cumple tres objetivos fundamentales:

1. Introducir la existencia del sistema de logros.
2. Mostrar al usuario dónde puede consultar sus logros.
3. Demostrar que los logros otorgan puntos de experiencia (XP).

Mediante este enfoque, el usuario aprende de manera práctica y progresiva sin necesidad de explicaciones teóricas. Además, la obtención de este logro inicial incluye una recompensa en XP, lo que evita que el usuario comience con 0 XP en la tabla de clasificación. Esto previene posibles sentimientos de inferioridad o desmotivación al compararse con otros usuarios más avanzados. El siguiente logro, “Realización del primer envío”, refuerza este aprendizaje al indicar claramente cuál es el siguiente paso en su interacción con el sistema. De este modo, se fomenta la exploración autónoma y se facilita la comprensión de las mecánicas básicas.

A continuación, se detalla la asignación de XP según las acciones realizadas por el usuario. La distribución de puntos, presentada en la Tabla 4.3, ha sido diseñada para:

- Reconocer todas las acciones relevantes (incluyendo los intentos no aceptados).
- Garantizar que las recompensas se perciban como significativas.
- Dificultar la obtención fraudulenta de XP, evitando que el sistema sea explotado de manera tediosa o artificial.

Acción	XP recibido
Envío no aceptado	1 XP
Envío aceptado	15 XP
Obtención de logro de bronce	20 XP
Obtención de logro de plata	40 XP
Obtención de logro de oro	60 XP

Table 4.3: Acciones y recompensas en XP

Sin embargo, según el sistema continúe creciendo y ante la posibilidad de que se redistribuyan o creen niveles de dominio, las recompensas podrán ser editadas.

4.3.3.3 Estética

El diseño del sistema de logros, estadísticas y puntos de experiencia (XP) busca evocar en los usuarios dos sensaciones fundamentales, alineadas con los principios de la **conciencia situacional** y la **motivación extrínseca**. Estas emociones no solo mejoran la experiencia del usuario, sino que también refuerzan su compromiso a largo plazo con la plataforma.

La conciencia situacional se refiere a la capacidad del usuario para comprender su posición actual dentro del sistema, así como las acciones necesarias para mejorar (Endsley y Jones (2012).) En este contexto, se logra mediante:

- Retroalimentación visual clara: Las estadísticas y los niveles de dominio proporcionan una imagen inmediata del progreso del usuario, permitiéndole identificar sus fortalezas y áreas de mejora.
- Objetivos definidos: Los logros actúan como hitos tangibles que guían al usuario hacia metas concretas, facilitando la planificación de su progreso.
- Contexto relevante: La información presentada (como la tabla de clasificación y los niveles de dominio) está directamente vinculada a los objetivos del usuario, evitando la sobrecarga de datos irrelevantes.

Esta conciencia permite al usuario no solo saber dónde se encuentra, sino también comprender qué acciones debe tomar para avanzar, fomentando una experiencia de aprendizaje autónoma y significativa.

La motivación extrínseca en este sistema se centra en el reconocimiento social y el aumento de estatus. Las estrategias clave incluyen:

- Logros y medallas: Actúan como símbolos de estatus que el usuario puede mostrar, generando un sentido de orgullo y pertenencia dentro de la comunidad.
- Tabla de clasificación: Fomenta la competencia saludable, especialmente en usuarios con perfiles hipercompetitivos, quienes buscan destacar incluso sin recompensas tangibles.
- Recompensas progresivas: Los puntos de experiencia (XP) y los niveles de dominio ofrecen una sensación de avance constante, reduciendo el riesgo de que el usuario dependa exclusivamente de "victorias" puntuales para mantener su motivación.

En conjunto, estas estrategias buscan crear una experiencia donde los usuarios se sientan capaces, reconocidos y motivados, equilibrando el deseo de superación personal con el disfrute del proceso.

4.4 Diseño de la interfaz

El primer paso para el diseño de la interfaz, es establecer los parámetros que guiarán el diseño de la interfaz, es decir, la guía de estilo. Los elementos de la guía de estilos se han elegido con la idea de mantener consistencia visual con la aplicación actual de *Acepta el Reto*, a la vez que se respetan las necesidades de contraste y accesibilidad:

1. Para la paleta de colores se han seleccionado 8 colores distintos: 3 tonos azules originarios de *¡Acepta el Reto!*, donde sirven para indicar selecciones, títulos o menús, un tono claro y un tono oscuro que pueden servir para color del fondo de la página y color de la letra o viceversa según qué tema se esté utilizando, y 3 tonos rojos obtenidos a partir de los tonos azules para representar situaciones de peligro o como color de resalte.
2. La tipografía es la misma que en *¡Acepta el Reto!*: Open Sans², tanto para el texto general como para títulos.
3. En cuanto a la iconografía, mantendremos la misma que en *¡Acepta el Reto!*. Se tratan de los iconos de Font Awesome³, los cuales son escalables y tienen una licencia libre.

En cuanto a los elementos de la interacción humano-máquina, tendremos en cuenta las heurísticas presentadas en Nielsen (1994). En específico para nuestra aplicación son importantes la visibilidad del estado del sistema, la equivalencia entre el sistema y el mundo real, y una representación de la información que simplifique el reconocimiento de los usuarios de la información, en vez de apelar a su memoria.

Manteniendo las heurísticas en mente, es hora de definir tanto las pantallas de nuestro sistema como su colocación de elementos para facilitar el uso (UX). Para

²<https://fonts.google.com/specimen/Open+Sans>

³<https://fontawesome.com/v3/>



Figure 4.3: Colores y topografía seleccionados para la guía de estilos

ello, se han realizado una serie de *wireframes*, una por cada pantalla de la aplicación. Las pantallas de la aplicación son:

- Inicio
- Estadísticas de usuario
 - Estadísticas
 - Medallero
- Estadísticas de un ejercicio
- Tabla de clasificación

TO DO : WIREFRAMES UX

TO DO: REPRESENTACIÓN DE LA INFORMACIÓN, DASHBOARDS

TO DO: DISEÑO DE LAS PLACAS DE LOGROS

4.5 Arquitectura del sistema

El diseño de la arquitectura del sistema ha sido ideado atendiendo a los requisitos no funcionales definidos para la aplicación en el apéndice A. En concreto, los requisitos de rendimiento (REQ-NF-001) y fiabilidad de los datos (REQ-NF-002) exigen la selección de tecnologías eficientes y robustas, capaces de garantizar un procesamiento ágil y una gestión segura de la información. Por otro lado, el requisito de usabilidad (REQ-NF-003) subraya la necesidad de implementar una interfaz y experiencia de usuario no solo intuitiva y personalizable, sino también mantenible y escalable a largo plazo.

Tras el análisis comparativo de las tecnologías disponibles, detallado en el capítulo 3, se ha decidido adoptar un *stack* tecnológico compuesto por React, Node.js y Redis para el desarrollo del *frontend*, *backend* y gestión de datos, respectivamente. Esta elección responde a las siguientes ventajas clave:

- React: Su capacidad para construir interfaces de usuario dinámicas y reactivas lo convierte en una herramienta idónea para el proyecto. El modelo basado en componentes no solo facilita la reutilización de código, sino que también optimiza la gestión del estado de la aplicación. Esto es especialmente relevante para cargar y visualizar datos en tiempo real, evitando recargas completas de página y reduciendo los tiempos de espera.
- Node.js: Destaca por su arquitectura orientada a eventos, que permite manejar operaciones asíncronas de manera eficiente. Esta característica es fundamental para garantizar un *backend* escalable y capaz de procesar múltiples solicitudes concurrentes sin degradar el rendimiento.
- Redis: Como base de datos *clave-valor* en memoria, su incorporación está motivada por la necesidad de optimizar el rendimiento mediante el almacenamiento en caché de datos de acceso frecuente. Redis es ideal para aplicaciones que requieren respuestas en tiempo real, como la nuestra, ya que reduce la latencia y aligera la carga sobre la base de datos principal.

La arquitectura actual de *¡Acepta el Reto!*, descrita en Pedro Pablo Gómez-Martín (2017), se basa en un modelo tradicional que emplea Tomcat como servidor de aplicaciones, PHP para el desarrollo del *frontend* y MySQL como sistema de gestión de bases de datos. Este enfoque, aunque funcional, presenta limitaciones en términos de escalabilidad, modularidad y experiencia de usuario, especialmente en comparación con las demandas actuales de las aplicaciones web.

En contraste, la arquitectura propuesta para este proyecto introduce un enfoque modular, lo que no solo simplifica la integración futura con la infraestructura existente de *¡Acepta el Reto!*, sino que también facilita la evolución independiente de cada componente. Esta modularidad, combinada con el uso de tecnologías modernas, asegura que el sistema sea adaptable, escalable y capaz de responder a las necesidades cambiantes del proyecto sin requerir rediseños profundos.

Chapter 5

Implementación

Todo lo referido al proceso de desarrollo, resolución de problemas, despliegue, etc.

Chapter 6

Evaluación

Esta sección tratará sobre qué sistemas implementamos para evaluar el trabajo desarrollado y qué resultados nos ha devuelto tras la implementación y el despliegue.

Conclusiones y Trabajo Futuro

Conclusiones del trabajo y líneas de trabajo futuro.

Antes de la entrega de actas de cada convocatoria, en el plazo que se indica en el calendario de los trabajos de fin de grado, el estudiante entregará en el Campus Virtual la versión final de la memoria en PDF.

Contribuciones Personales

Conviene escribir sobre las siguientes cosas: - Una reflexión sobre la elección del tema, los métodos de investigación utilizados, los desafíos superados. - Qué aportes originales se han hecho tanto al trabajo, como a la literatura existente. Qué ideas originales se han desarrollado durante el proceso o qué innovaciones metodológicas se han puesto en práctica. - El desarrollo de habilidades en investigación, desarrollo de aplicaciones web, y comunicación escrita - Reflexión sobre el impacto personal y profesional a raíz del desarrollo del trabajo - Posibles líneas de investigación futuras a raíz del desarrollo del trabajo

Néstor García Mayor

María Pajares Vázquez

Inés Triviño Rello

Bibliography

*El que lee mucho y anda mucho ve mucho y
sabe mucho*

Miguel de Cervantes Saavedra

BARTLE, R. A. Players who suit muds. 1996. Consultado: 2026-01-24.

BULJIC, I., KADUSIC, E., CVIJANOVIC, T., HADZAJLIC, N. y ZIVIC, N. Comparative performance analysis of leading backend frameworks for developers. En *2025 24th International Symposium INFOTEH-JAHORINA (INFOTEH)*, página 1–5. IEEE, East Sarajevo, Bosnia and Herzegovina, 2025. ISBN 9798331515799.

ENDSLEY, M. R. y JONES, D. G. *Designing for situation awareness: an approach to user-centered design*. CRC Pr, Boca Raton, Fla., 2. ed edición, 2012. ISBN 9781420063585.

FORSYTHE, G. E. y WIRTH, N. Automatic grading programs. *Communications of the ACM*, vol. 8(5), página 275–278, 1965. ISSN 0001-0782, 1557-7317.

KUO, J.-Y., WEN, Z.-J., HSIEH, T.-F. y HUANG, H.-X. A study on the security of online judge system applied sandbox technology. *Electronics*, vol. 12(14), página 3018, 2023. ISSN 2079-9292.

LEI, K., MA, Y. y TAN, Z. Performance comparison and evaluation of web development technologies in php, python, and node.js. En *2014 IEEE 17th International Conference on Computational Science and Engineering*, página 661–668. IEEE, Chengdu, China, 2014. ISBN 9781479979813.

MORSCHHEUSER, B., HASSAN, L., WERDER, K. y HAMARI, J. How to design gamification? a method for engineering gamified software. *Information and Software Technology*, vol. 95, página 219–237, 2018. ISSN 09505849.

NIELSEN, J. Enhancing the explanatory power of usability heuristics. En *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*, página 152–158. ACM, Boston Massachusetts USA, 1994. ISBN 9780897916509.

- OLSINA, L., LAFUENTE, G. y ROSSI, G. *Specifying Quality Characteristics and Attributes for Websites*, vol. 2016, páginas 266–278. Springer Berlin Heidelberg, Berlin, Heidelberg, 2001. ISBN 9783540421306.
- PEDRO PABLO GÓMEZ-MARTÍN, M. A. G.-M. ¡acepta el reto!: juez online para docencia en español. *Actas de las Jenui*, vol. 2, página 289–296, 2017.
- PRESSMAN, R. S. *Ingeniería del software: un enfoque práctico*. McGraw-Hill, México, 7a ed. edición, 2010. ISBN 9786071503145.
- REVILLA, M. , MANZOOR, S. y LIU, R. Competitive learning in informatics: The uva online judge experience. 2008.
- SEKHAR EMMANNI, P. Comparative analysis of angular, react, and vue.js in single page application development. *International Journal of Science and Research (IJSR)*, vol. 12(6), página 2971–2974, 2023. ISSN 23197064.
- WASIK, S., ANTCHAK, M., BADURA, J., LASKOWSKI, A. y STERNAL, T. A survey on online judge systems and their applications. *ACM Computing Surveys*, vol. 51(1), página 1–34, 2019. ISSN 0360-0300, 1557-7341.
- WATANOBE, Y., RAHMAN, M. M., MATSUMOTO, T., RAGE, U. K. y RAVIKUMAR, P. Online judge system: Requirements, architecture, and experiences. *International Journal of Software Engineering and Knowledge Engineering*, vol. 32(06), página 917–946, 2022. ISSN 0218-1940, 1793-6403.
- ZICHERMANN, G. y CUNNINGHAM, C. *Gamification by design: implementing game mechanics in web and mobile apps*. O'Reilly, Beijing, Köln, 2011. ISBN 9781449397678.

Especificación de Requisitos Software

Versión 0.1

Preparada por Inés Triviño Rello

Universidad Complutense de Madrid

A 31 de enero de 2026

A.1 Introducción

A.1.1 Propósito del documento

El propósito del presente documento es servir como especificación de requisitos de software (SRS) para las funcionalidades de estadísticas y logros del juez on-line *¡Acepta el reto!*. Este documento constituye la base para el diseño, desarrollo, validación y mantenimiento de dichas funcionalidades.

El documento está dirigido principalmente a los desarrolladores y mantenedores de las funcionalidades descritas, a los mantenedores del juez *¡Acepta el reto!*, y a cualquier otra persona interesada en comprender el funcionamiento, alcance y limitaciones del producto.

Asimismo, este documento debe emplearse como referencia durante el ciclo de vida del software para constatar las funcionalidades implementadas, las decisiones de diseño adoptadas, los motivos que las justifican y las limitaciones actuales del sistema, con el objetivo de facilitar su mantenibilidad y evolución futura.

A.1.2 Ámbito del producto

El producto no cuenta con una denominación propia. A lo largo de este documento se hará referencia a él como *las funcionalidades de estadísticas y logros para el juez ¡Acepta el reto!*, manteniendo la consistencia con el título del proyecto en cuyo marco se desarrolla.

En este documento se describen las especificaciones correspondientes a la primera versión del producto. Las capacidades principales incluyen la visualización fiel y

actualizada de estadísticas de usuario, estadísticas asociadas a los ejercicios, y la gestión de logros o medallas otorgadas por la realización de determinadas acciones.

Cabe destacar que este documento se refiere exclusivamente a las funcionalidades de estadísticas y logros, las cuales se implementan de manera independiente a la arquitectura actual de *¡Acepta el reto!*. No obstante, el producto se diseña con el objetivo de facilitar una posible integración futura en la arquitectura del juez, en caso de que se considere oportuno.

A.1.3 Definiciones, acrónimos y abreviaciones

- **UI:** User Interface.
- **API:** Application Programming Interface.
- **SRS:** Software Requirements Specification.
- **TFG:** Trabajo Fin de Grado.
- **¡Acepta el reto!:** Juez online de programación competitiva objeto de este proyecto.

A.1.4 Referencias

- IEEE Std 830-1998 / IEEE Std 29148-2018, *Software Requirements Specification*.

A.2 Descripción general del producto

A.2.1 Perspectiva del producto

El producto se desarrolla en el marco de un Trabajo de Fin de Grado (TFG), bajo la supervisión de los creadores y mantenedores de *¡Acepta el reto!*. Su objetivo es cubrir la necesidad de introducir mayor dinamismo y nuevas fuentes de motivación para los usuarios del juez online mediante estadísticas detalladas y un sistema de logros.

Aunque la integración directa de estas funcionalidades en el juez no forma parte del alcance del proyecto, el diseño tendrá en cuenta criterios de modularidad y desacoplamiento que faciliten una posible integración futura.

A.2.2 Funciones del producto

- Mostrar estadísticas agregadas y detalladas de los usuarios que reflejen su progreso y actividad.
- Mostrar estadísticas de ejercicios que permitan inferir su dificultad, popularidad o tasa de éxito.

- Actualizar las estadísticas de manera automática y en tiempo cercano al real.
- Otorgar logros o medallas a los usuarios como recompensa por acciones específicas o hitos alcanzados.
- Ordenar a los usuarios en una tabla de clasificación en base a su puntuación.

A.2.3 Limitaciones del producto

El producto estará sujeto a limitaciones técnicas derivadas del stack tecnológico seleccionado durante el desarrollo del TFG, así como a restricciones de tiempo y recursos propios de un proyecto académico. Asimismo, el sistema dependerá de la disponibilidad y calidad de los datos proporcionados por el juez *¡Acepta el reto!* u otras fuentes externas. No se contemplan requisitos específicos de hardware más allá de los necesarios para ejecutar una aplicación web moderna.

A.2.4 Características de los usuarios

Se identifican los siguientes grupos de usuarios:

- **Usuarios finales:** estudiantes o programadores que utilizan el juez. Tienen conocimientos técnicos básicos, acceden con frecuencia al sistema y su objetivo principal es medir su progreso y obtener motivación adicional para mejorar sus habilidades de programación y algoritmia.
- **Administradores/mantenedores:** usuarios con conocimientos técnicos avanzados y acceso a funciones de configuración o mantenimiento. Su uso es esporádico y orientado a la supervisión del sistema.

No se prevén necesidades especiales de accesibilidad en esta primera versión, aunque se seguirán buenas prácticas de diseño de interfaces.

A.2.5 Supuestos y dependencias

Se asume la disponibilidad de datos fiables sobre usuarios y ejercicios, así como el acceso a interfaces o mecanismos que permitan obtenerlos. El producto depende de librerías y servicios de terceros para la persistencia de datos y la visualización de estadísticas. Si alguna de estas dependencias no se cumple, el impacto principal será la imposibilidad de mostrar estadísticas actualizadas o de calcular correctamente los logros.

A.3 Requisitos

A.3.1 Interfaces externas

El sistema expone y consume las siguientes interfaces externas:

- **Interfaces de usuario (UI)** El sistema proporcionará una interfaz web accesible mediante navegadores modernos (Chrome, Firefox, Edge, etc). La UI incluirá:
 - Paneles de estadísticas de usuario.
 - Vistas de estadísticas de ejercicios.
 - Sección de logros y medallas obtenidas.
 - Tabla de clasificación de usuarios.Se seguirán principios de diseño responsivo y consistencia visual con el estilo general de *¡Acepta el reto!*.
- **Interfaces software (API)** El sistema consumirá datos provenientes de las APIs del juez que proporcionen información sobre usuarios, envíos y ejercicios.
- **Interfaces de hardware** No se definen interfaces hardware específicas. El sistema se ejecutará sobre infraestructura estándar de servidores web y será accesible desde dispositivos de escritorio y móviles.

A.3.2 Funcionales

REQ-FUN-001

- **Título:** Visualización de estadísticas de usuario
- **Descripción:** El sistema debe permitir visualizar estadísticas asociadas a un usuario, incluyendo número de ejercicios resueltos, envíos realizados y tasa de aciertos, entre otros.
- **Racional:** Proporcionar al usuario información clara sobre su progreso y actividad.
- **Criterio de aceptación:** Dado un usuario válido, el sistema muestra sus estadísticas de forma correcta y consistente.
- **Método de verificación:** Test funcional.

REQ-FUN-002

- **Título:** Visualización de estadísticas de ejercicios
- **Descripción:** El sistema debe mostrar estadísticas agregadas por ejercicio, como número de intentos, número de resoluciones y porcentaje de éxito.
- **Racional:** Ayudar a los usuarios a estimar la dificultad o popularidad de los ejercicios.
- **Criterio de aceptación:** Las estadísticas mostradas coinciden con los datos almacenados.
- **Método de verificación:** Test funcional.

REQ-FUN-003

- **Título:** Actualización automática de estadísticas y puntos
- **Descripción:** El sistema debe actualizar las estadísticas y puntos de usuario cuando se produzcan nuevos eventos relevantes (por ejemplo, un envío evaluado).
- **Racional:** Garantizar que la información mostrada sea actual y fiable.
- **Criterio de aceptación:** Las estadísticas reflejan los cambios sin necesidad de intervención manual.
- **Método de verificación:** Demostración, test funcional.

REQ-FUN-004

- **Título:** Gestión de logros
- **Descripción:** El sistema debe otorgar logros a los usuarios cuando se cumplan determinadas condiciones predefinidas.
- **Racional:** Incrementar la motivación de los usuarios mediante recompensas simbólicas.
- **Criterio de aceptación:** Un logro se asigna automáticamente cuando se cumple su condición.
- **Método de verificación:** Test funcional.

REQ-FUN-005

- **Título:** Consulta de logros obtenidos
- **Descripción:** El sistema debe permitir a un usuario consultar la lista de logros que ha obtenido.
- **Racional:** Hacer visibles las recompensas y fomentar la participación.
- **Criterio de aceptación:** El usuario puede ver todos sus logros sin errores.
- **Método de verificación:** Demostración y test funcional.

REQ-FUN-006

- **Título:** Actualización de la tabla de clasificación
- **Descripción:** El sistema debe actualizar automáticamente la tabla de clasificación según las puntuaciones de los usuarios.
- **Racional:** Mostrar una imagen fiel y actualizada de las puntuaciones de los usuarios.

- **Criterio de aceptación:** La tabla de clasificación se actualiza correctamente cada vez que un usuario completa una acción que afecta su puntuación.
- **Método de verificación:** Test funcional.

REQ-FUN-007

- **Título:** Consulta de la tabla de clasificación
- **Descripción:** El sistema debe permitir a los usuarios consultar la tabla de clasificación, mostrando su posición y la de otros usuarios según su puntuación.
- **Racional:** Permitir a los usuarios compararse con otros.
- **Criterio de aceptación:** El usuario puede visualizar la tabla de clasificación completa, incluyendo su posición, sin errores y con los datos actualizados.
- **Método de verificación:** Demostración y test funcional.

REQ-FUN-008

- **Título:** Compartición de estadísticas y logros
- **Descripción:** El sistema debe permitir a los usuarios compartir sus estadísticas o logros de usuario como imagen.
- **Racional:** Permitir a los usuarios compartir sus hitos, generando más motivación.
- **Criterio de aceptación:** El usuario puede recibir desde la aplicación una imagen o link con la que mostrar una estadística o logro recibido a otras personas.
- **Método de verificación:** Demostración.

A.3.3 No funcionales

REQ-NF-001

- **Título:** Rendimiento
- **Descripción:** El sistema debe responder a las consultas de estadísticas en menos de 2 segundos en condiciones normales de carga.
- **Racional:** Garantizar una experiencia de usuario fluida.
- **Criterio de aceptación:** El tiempo de respuesta se mantiene dentro del límite establecido.
- **Método de verificación:** Test de rendimiento.

REQ-NF-002

- **Título:** Fiabilidad de los datos
- **Descripción:** Las estadísticas mostradas deben ser consistentes con los datos de origen.
- **Racional:** Evitar información incorrecta que afecte a la confianza del usuario.
- **Criterio de aceptación:** No se detectan discrepancias en auditorías de datos.
- **Método de verificación:** Análisis.

REQ-NF-003

- **Título:** Usabilidad
- **Descripción:** La interfaz debe ser comprensible sin necesidad de formación previa.
- **Racional:** Facilitar la adopción del sistema por parte de usuarios sin experiencia.
- **Criterio de aceptación:** Usuarios de prueba completan tareas básicas sin ayuda.
- **Método de verificación:** Test de usabilidad.

A.3.4 Diseño e implementación

El sistema deberá diseñarse siguiendo principios de modularidad y desacoplamiento para facilitar su mantenimiento y posible integración futura.

Se priorizará el uso de tecnologías web estándar y ampliamente soportadas.

El sistema deberá ser portable entre navegadores modernos y fácilmente desplegable en entornos de servidor convencionales.

Las decisiones de diseño deberán documentarse para facilitar la mantenibilidad y reutilización del código.

A.4 Verificación

La verificación del sistema se llevará a cabo comprobando que cada requisito definido en la Sección 3 ha sido satisfecho de forma objetiva. Para ello, se emplearán distintos métodos de verificación, incluyendo pruebas funcionales, análisis y demostraciones.

Cada requisito estará asociado a uno o más casos de prueba que permitan validar su cumplimiento. Los resultados de la verificación se documentarán, indicando si el requisito ha sido cumplido, parcialmente cumplido o no cumplido, así como las incidencias detectadas y las acciones correctivas adoptadas.

Appendix B

Formulario sobre jugadores

En este apéndice que muestran las preguntas propuestas en el formulario de estudio de los jugadores junto con las respuestas obtenidas.

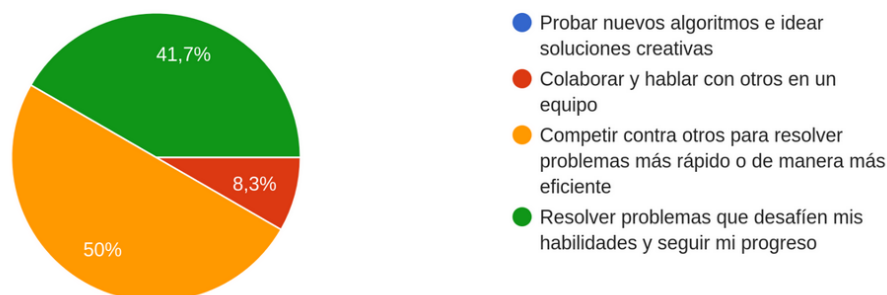
En total, el formulario fue respondido por 12 usuarios diferentes. El formulario se compartió en el grupo de la asignatura de Programación Competitiva de la Facultad de Informática de la Universidad Complutense de Madrid, aunque se instó a los encuestados a compartirlo con otras personas de interés, por lo que no se puede asegurar que todas las respuestas procedan de estudiantes de la asignatura mencionada.

Las preguntas del formulario están dirigidas a saber, cuál es el tipo de jugador para el que estamos diseñando (siguiendo la taxonomía de jugadores propuesta por Bartle (1996)), qué uso hacen los encuestados de jueces en línea, y qué tipo de funcionalidades buscan estos jugadores.

B.1 Perfil del jugador

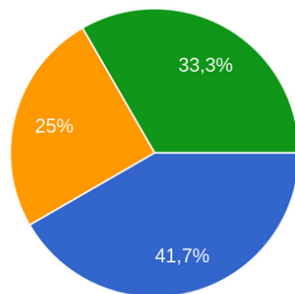
Cuando participo en competiciones de programación, prefiero:

12 respuestas



Practicando programación competitiva disfruto:

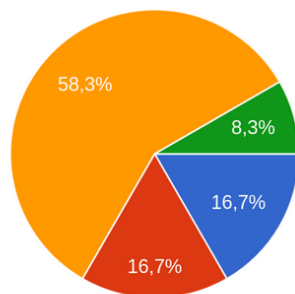
12 respuestas



- Descubriendo nuevas técnicas o trucos para resolver problemas
- Discutiendo nuevas estrategias con otros en fórums o chats
- Compitiendo contra otros y viendo mi ránking mejorar
- Completando problemas especialmente difíciles y superando mis logros anteriores

Cuando me enfrento a un problema de programación difícil:

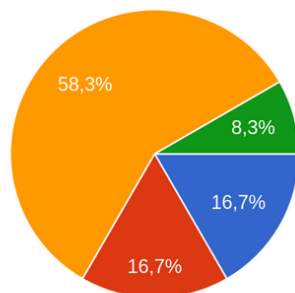
12 respuestas



- Investigo nuevos algoritmos o técnicas que me puedan ayudar a resolverlo
- Pido ayuda o colaboro con otros para encontrar una solución
- Me concentro en resolverlo solo, aunque me lleve tiempo, para demostrar mi habilidad
- Trabajo en problemas más sencillos para mejorar mis habilidades antes de intentarlo de nuevo

Cuando me enfrento a un problema de programación difícil:

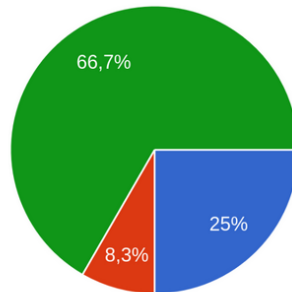
12 respuestas



- Investigo nuevos algoritmos o técnicas que me puedan ayudar a resolverlo
- Pido ayuda o colaboro con otros para encontrar una solución
- Me concentro en resolverlo solo, aunque me lleve tiempo, para demostrar mi habilidad
- Trabajo en problemas más sencillos para mejorar mis habilidades antes de intentarlo de nuevo

Me siento más satisfecho cuando:

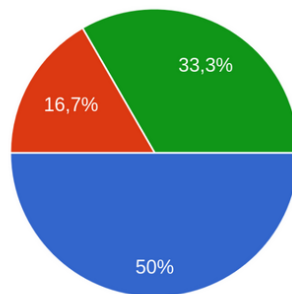
12 respuestas



- Aprendo nuevos algoritmos o resuelvo problemas de manera innovadora
- Ayudo a otros al explicar conceptos o revisar su código
- Ocupo un puesto alto en las clasificaciones o gano concursos
- Resuelvo un problema difícil y noto un progreso tangible en mis habilidades

Me gusta que me reconozcan por:

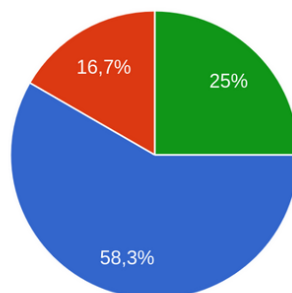
12 respuestas



- Mi capacidad para descubrir soluciones eficientes o únicas a problemas complejos
- Mi habilidad para ayudar a otros compartiendo soluciones o explicaciones
- Mi destreza para competir y resolver problemas más rápido que los demás
- Mis logros al resolver problemas difíciles y mejorar mis habilidades constantem...

Si pudiera, me gustaría:

12 respuestas

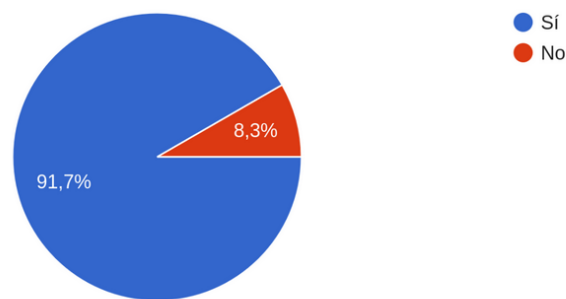


- Explorar diferentes tipos de problemas y algoritmos para ampliar mis conocimientos
- Compartir mis soluciones y conocimientos con la comunidad para ayudar a los demás a aprender
- Competir en todos los desafíos, buscando superarme en cada uno
- Mejorar continuamente mis habilidades de programación, dominando todos lo...

B.2 Uso de jueces en línea

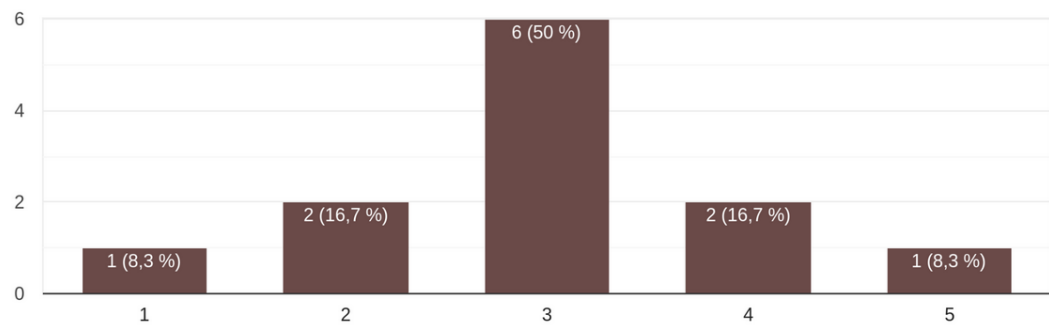
¿Alguna vez has participado en algún concurso o clase de programación competitiva?

12 respuestas



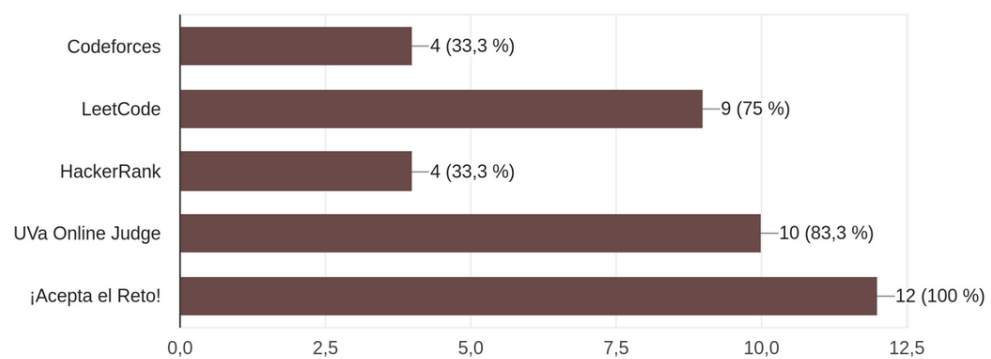
¿Cómo calificarías tu nivel en programación competitiva?

12 respuestas



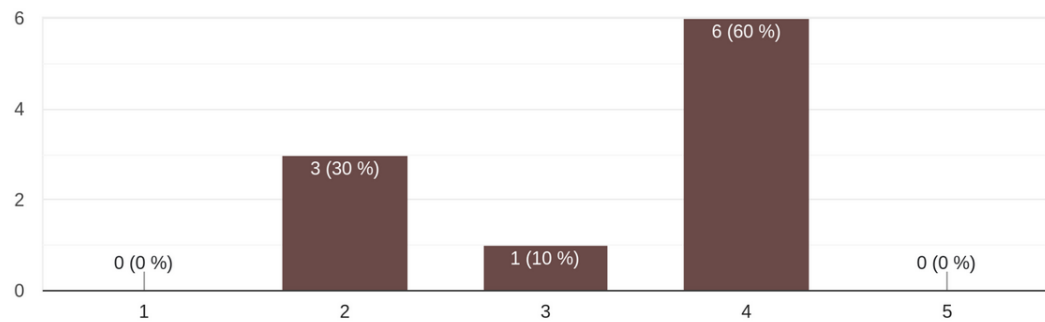
¿Cuáles de estos jueces en línea conoces? (Elige todos los que correspondan)

12 respuestas



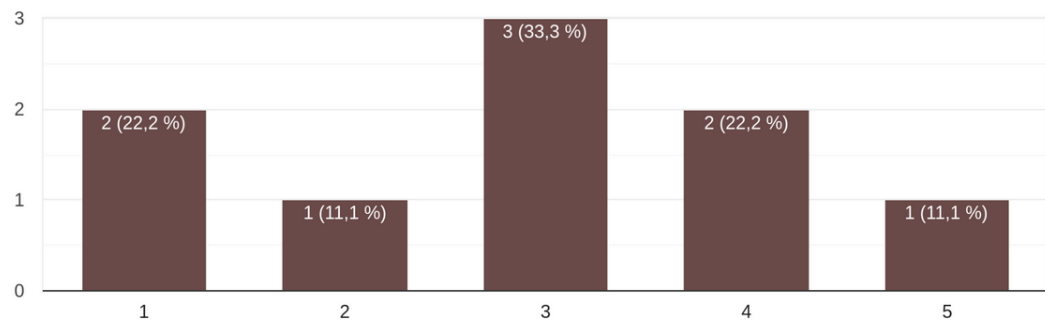
¿Cómo calificarías tu nivel de satisfacción con las estadísticas de usuario que proporcionan estos jueces? (no contestar si no se usan)

10 respuestas



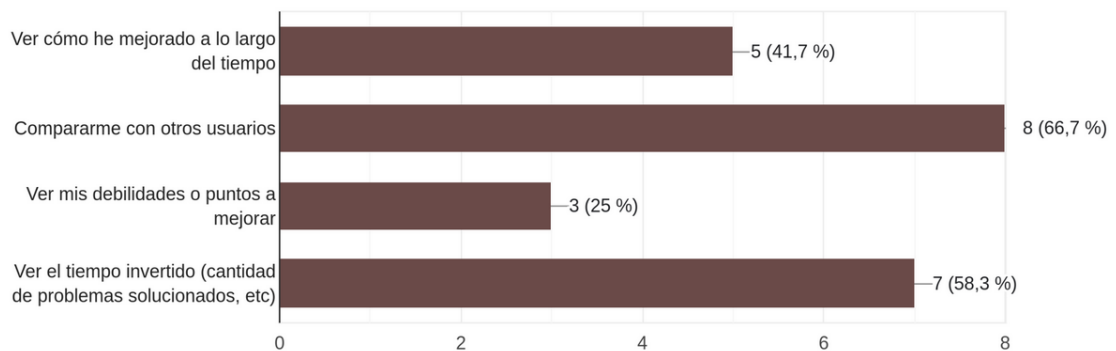
¿Cómo calificarías tu nivel de satisfacción con los logros / badges que proporcionan estos jueces? (no contestar si no se usan)

9 respuestas



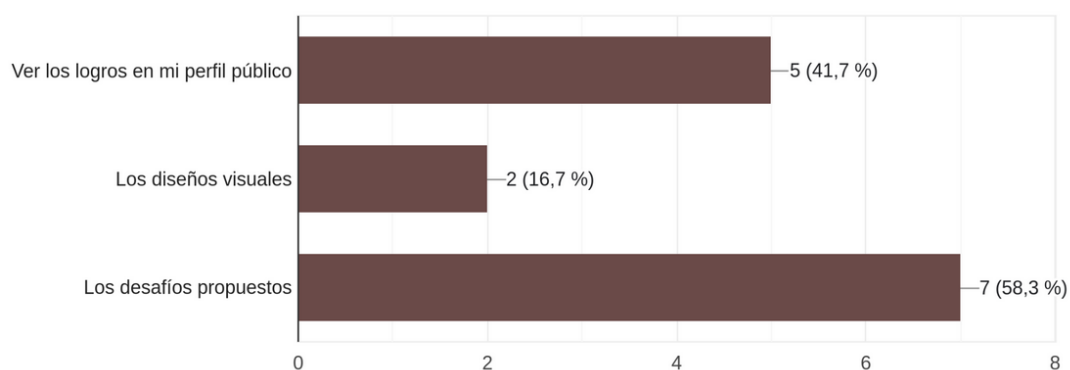
¿Qué es lo que más te gusta de las estadísticas de usuario que ofrecen los jueces en línea actuales?

12 respuestas



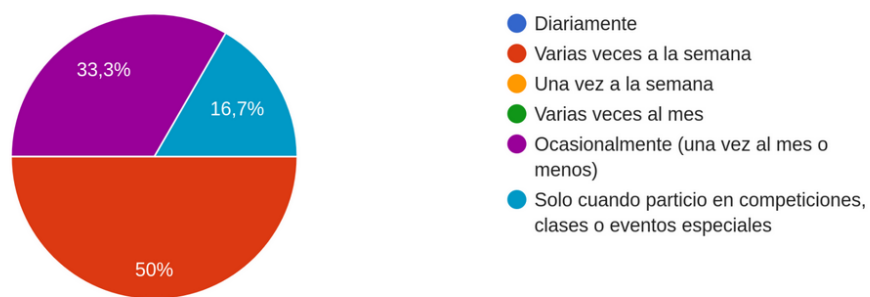
¿Qué es lo que más te gusta de los logros e insignias que ofrecen los jueces en línea actuales?

12 respuestas



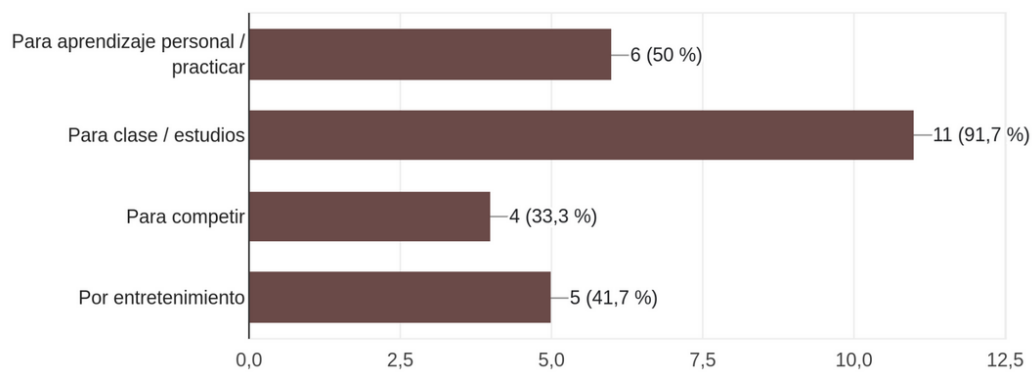
¿Con qué frecuencia usas jueces de programación en línea?

12 respuestas



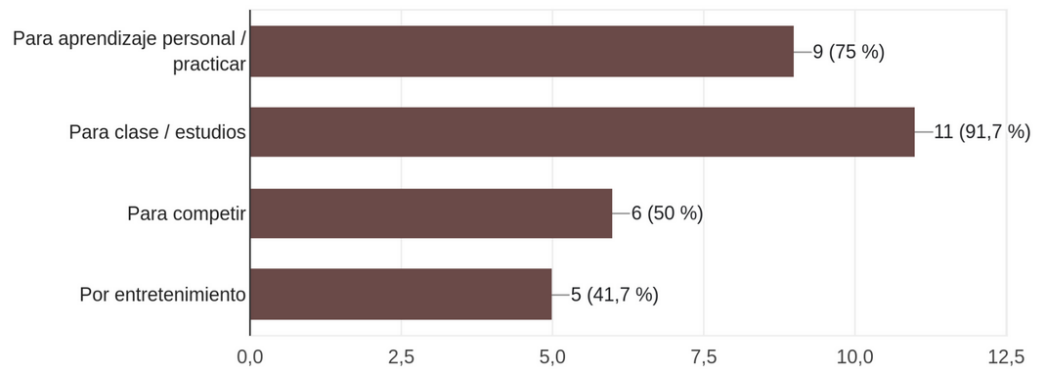
¿En qué contexto usas jueces de programación en línea?

12 respuestas



¿Para qué usarías o has usado Acepta el Reto?

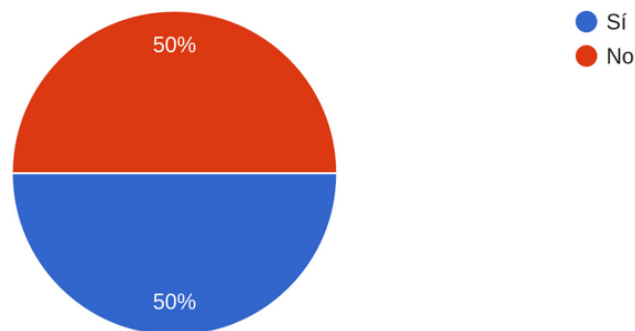
12 respuestas



B.3 Preferencias de recompensas y motivaciones

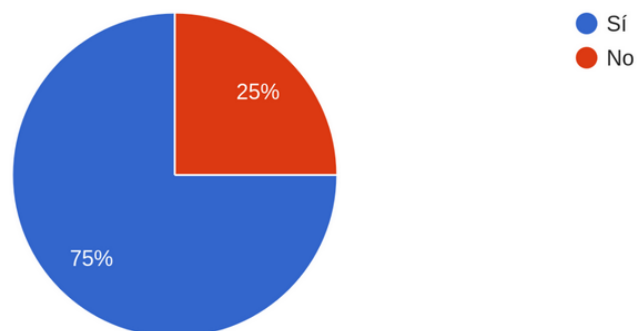
¿Estás satisfecho con tu nivel y consistencia de práctica en programación competitiva?

12 respuestas



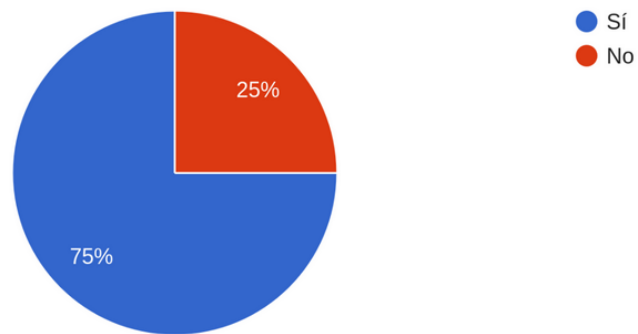
¿Crees que recibir retroalimentación constante sobre tu progreso te motivaría más?

12 respuestas



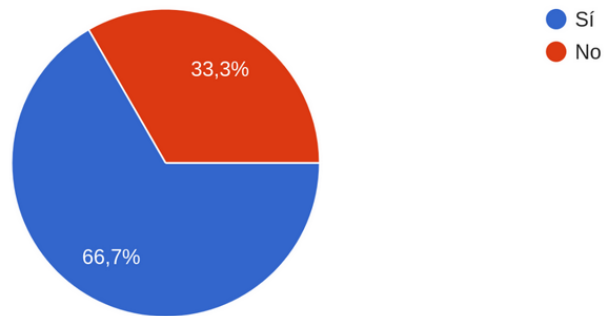
¿Crees que recibir retroalimentación constante sobre tu progreso te motivaría más?

12 respuestas



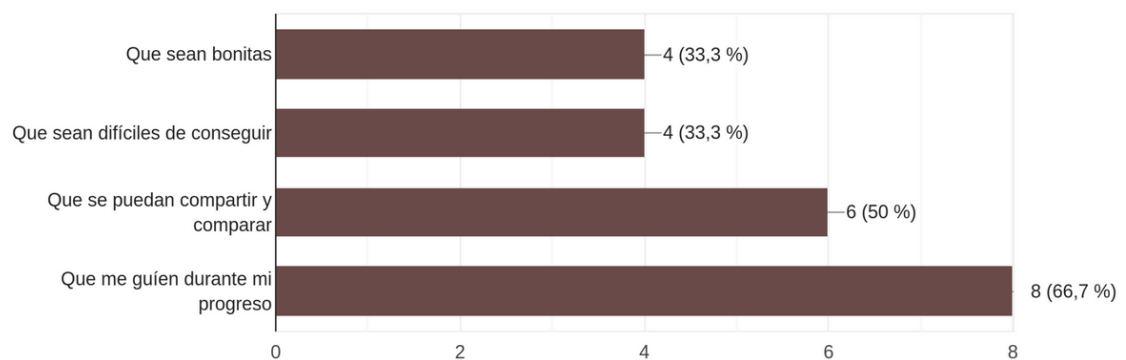
¿Crees que poder compartir y comparar tu progreso con otras personas te motivaría más?

12 respuestas



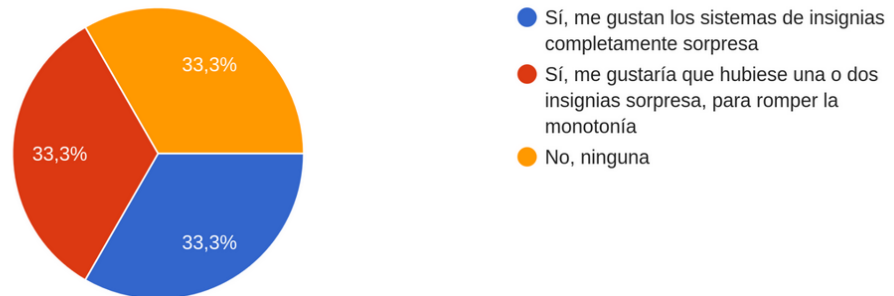
¿Qué te gustaría de un sistema de insignias?

12 respuestas



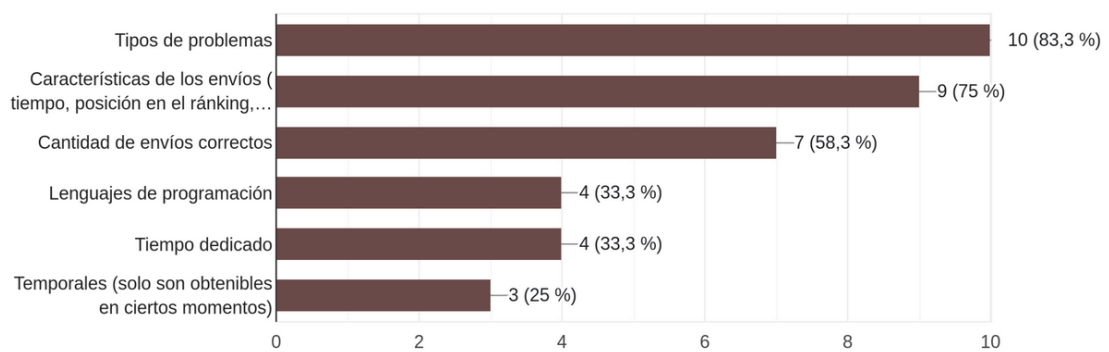
¿Te gustan las insignias "sorpresa" (recompensas por logros secretos, se consiguen sin esperarlas)?

12 respuestas



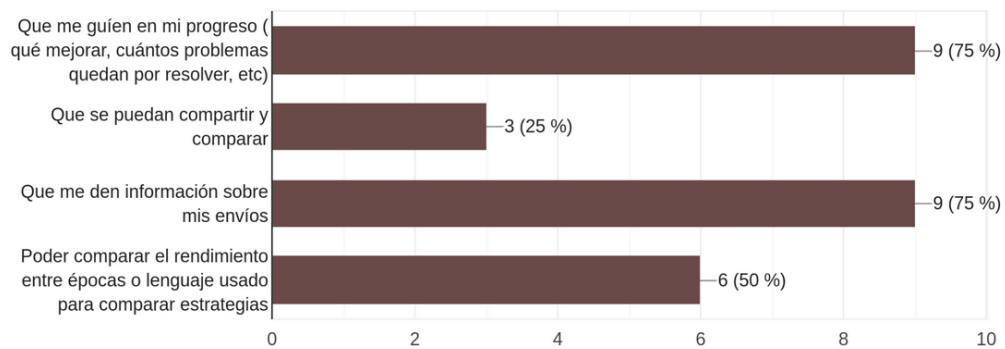
¿En qué tipo de categorías te gustaría que se enfocasen las insignias?

12 respuestas



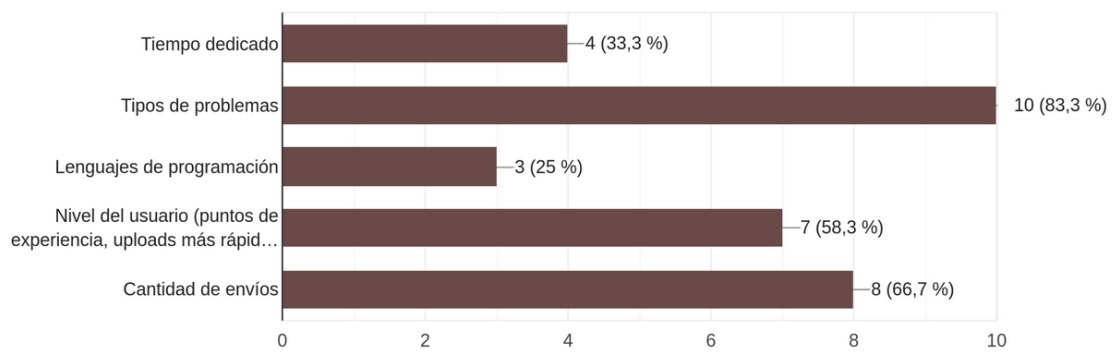
¿Qué te gustaría de una pantalla de estadísticas de usuario?

12 respuestas



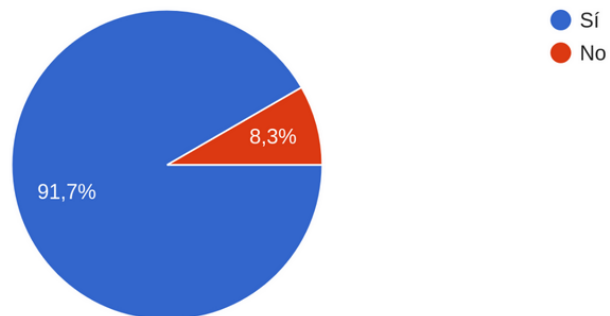
¿En qué tipo de categorías te gustaría que se enfocasen las estadísticas?

12 respuestas



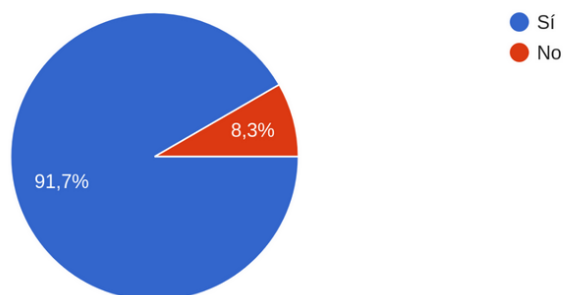
¿Te gustaría salir en una tabla de clasificación en base a tus puntos de experiencia?

12 respuestas



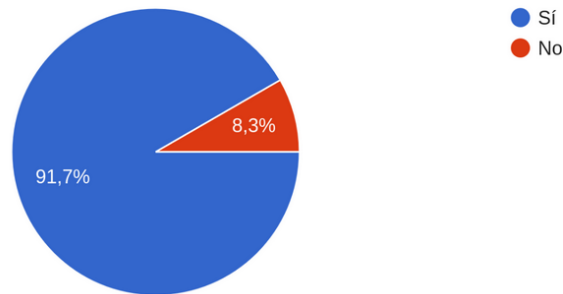
¿Te gustaría poder filtrar la tabla de clasificación por niveles (compararte con usuarios de nivel similar)?

12 respuestas



¿Te gustaría poder filtrar la tabla de clasificación por tipos de problemas (compararte con otros usuarios en un campo de conocimiento específico)?

12 respuestas



B.4 Información extra y retroalimentación

Información extra y retroalimentación

¿Tienes algo que comentar sobre los sistemas de tablas de clasificación, estadísticas, y logros / insignias (cosas que no te gusten, que quisieras ver, etc)?

2 respuestas

Hazlo ranked y que subas de rank

Quizás se podría introducir en que porcentaje de usuario entras por velocidad de solución

¿Tienes algo que comentar sobre formulario o retroalimentación para nosotros?

1 respuesta

Os recomiendo mirar la página de kattis, tienen un sistema de grupos/clubs y tienen unas estadísticas interesantes

Appendix C

Casos de Prueba

Nota: Quizás esto debería ser para la guía de estilo, o todas las imágenes de wireframes, o algo así.

– ¡Cuarenta y dos! – chilló Loonquawl –.

¿Eso es todo lo que tienes que decirnos después de siete millores y medio de años de trabajo?

– Lo he comprobado con mucho cuidado – manifestó el ordenador –,
y ésta es exactamente la respuesta.

Para ser franco con vosotros, creo que el problema consiste en que nunca habéis sabido realmente cuál es la pregunta.

Guía del autoestopista galáctico
Douglas Adams

